

RÉPUBLIQUE DÉMOCRATIQUE DU CONGO

Université de Kinshasa

Faculté des Sciences

Département de Mathématiques et Informatique

— *Travail de Fin d'Études* —

Mémoire de Licence présenté en vue de l'obtention du grade de Licencié en Sciences Informatiques

**Développement d'une plateforme web basée
sur l'Intelligence Artificielle pour la
détection du plagiat académique**

Cas pilote : Faculté des Sciences de l'Université de Kinshasa

Présenté par :	Moïse KASOMBO
Travail de Fin d'Études :	Mémoire de Licence
Département :	Mathématiques et Informatique
Encadré par :	【Professeur encadreur】
Cas pilote :	Faculté des Sciences — UNIKIN

Année académique : 2024 - 2025

Kinshasa, **【Année académique】**

TABLE DES MATIÈRES

INTRODUCTION GÉNÉRALE.....	3
I.1. Préambule.....	3
I.2. Problématique.....	3
I.3. Hypothèse du travail.....	4
I.4. Choix et intérêt du sujet.....	5
I.5. Délimitation du sujet.....	5
I.5.1. Délimitation dans le temps.....	6
I.5.2. Délimitation dans l'espace.....	6
I.5.3. Délimitation du contenu.....	6
I.6. Méthodes et techniques utilisées.....	6
I.6.1. Méthodes d'analyse et de conception.....	7
I.6.2. Méthodologie data science : CRISP-DM.....	7
I.6.3. Techniques de collecte de données.....	7
I.6.4. Techniques d'évaluation.....	8
I.7. Subdivision du travail.....	8
CHAPITRE I — FONDEMENTS DE L'INTELLIGENCE ARTIFICIELLE	
APPLIQUÉE À LA DÉTECTION AUTOMATIQUE DU PLAGIAT.....	8
I.1. Introduction.....	9
I.2. Intelligence Artificielle : définition et évolution.....	9
I.3. Machine Learning.....	10
I.4. Deep Learning.....	10
I.5. Traitement Automatique du Langage Naturel (NLP).....	11
I.6. Embeddings vectoriels.....	11

I.7. Similarité cosinus et métriques de distance.....	12
I.8. Recherche vectorielle.....	13
I.9. IA générative et grands modèles de langage.....	13
I.10. État de l'art de la détection automatique du plagiat par IA.....	14
I.11. Conclusion.....	15
CHAPITRE II — ÉTAT DE L'ART DU PLAGIAT ACADÉMIQUE ET ANALYSE DES BESOINS.....	15
II.1. Introduction.....	15
II.2. Définition du plagiat.....	16
II.3. Quelques domaines du plagiat.....	16
II.3.1. Dans les sciences.....	17
II.3.2. Dans le journalisme.....	17
II.3.3. Dans la littérature et les arts.....	17
II.3.4. Dans le monde académique.....	17
II.4. Types de plagiat.....	18
II.5. Détection automatique du plagiat.....	18
II.6. Causes et facteurs du plagiat académique.....	19
II.6.1. Facteurs individuels.....	19
II.6.2. Facteurs institutionnels.....	19
II.6.3. Facteurs technologiques.....	20
II.7. Conséquences du plagiat.....	20
II.8. Mesures de lutte contre le plagiat.....	21
II.8.1. Mesures pédagogiques.....	21
II.8.2. Mesures institutionnelles.....	21
II.8.3. Mesures techniques.....	21
II.9. Plagiat assisté par IA générative.....	22

II.10. Techniques modernes de paraphrase automatique.....	22
II.11. Limites des logiciels classiques.....	23
II.12. Comparaison avec les plateformes modernes.....	23
II.13. Conclusion.....	24
CHAPITRE III — CONCEPTION D'UNE PLATEFORME WEB INTELLIGENTE BASÉE SUR L'IA.....	24
III.1. Introduction.....	24
III.2. Présentation du cadre pilote : la Faculté des Sciences.....	25
III.3. Analyse des besoins fonctionnels.....	25
III.3.1. Gestion des utilisateurs et authentification.....	26
III.3.2. Gestion académique.....	26
III.3.3. Dépôt et gestion des travaux.....	26
III.3.4. Analyse IA et détection.....	26
III.3.5. Rapports et tableau de bord.....	27
III.3.6. Historique et traçabilité.....	28
III.4. Analyse des besoins non fonctionnels.....	28
III.5. Architecture générale de la plateforme.....	28
III.5.1. Couche présentation (frontend).....	29
III.5.2. Couche application (backend).....	29
III.5.3. Couche services IA.....	29
III.5.4. Couche données.....	29
III.6. Architecture modulaire.....	30
III.7. Pipeline de traitement IA.....	30
III.7.1. Soumission du document.....	30
III.7.2. Extraction du texte.....	31
III.7.3. Prétraitement NLP.....	31

III.7.4. Génération des embeddings.....	31
III.7.5. Recherche vectorielle.....	31
III.7.6. Calcul de similarité.....	32
III.7.7. Détection et classification.....	32
III.7.8. Génération du rapport.....	32
III.8. Base documentaire relationnelle.....	32
III.9. Base vectorielle.....	33
III.10. Conception de l'API REST.....	33
III.11. Modélisation UML.....	34
III.11.1. Diagramme de cas d'utilisation.....	34
III.11.2. Diagramme de classes.....	34
III.11.3. Diagramme de séquences.....	35
III.12. Diagrammes d'architecture logicielle.....	35
III.13. Conclusion.....	35
CHAPITRE IV — IMPLÉMENTATION, EXPÉRIMENTATION ET	
ÉVALUATION.....	36
IV.1. Introduction.....	36
IV.2. Stack technique.....	36
IV.3. Frontend.....	37
IV.4. Backend.....	37
IV.5. Base relationnelle.....	38
IV.6. Base vectorielle.....	41
IV.7. OCR et extraction du texte.....	42
IV.8. Pipeline NLP.....	45
IV.9. Génération des embeddings.....	49
IV.10. Recherche vectorielle.....	53

IV.11. Calcul de similarité et scoring.....	58
IV.12. Détection des paraphrases.....	61
IV.13. Génération automatique des rapports PDF.....	64
IV.14. Tests.....	64
IV.15. Évaluation sur corpus pilote.....	65
IV.15.1. Constitution du corpus pilote.....	65
IV.15.2. Baseline lexicale.....	65
IV.15.3. Métriques d'évaluation.....	65
IV.15.4. Résultats quantitatifs.....	65
IV.15.5. Analyse par type de plagiat.....	66
IV.16. Performances et scalabilité.....	67
IV.17. Limites observées.....	67
IV.18. Conclusion.....	68
CONCLUSION GÉNÉRALE.....	68
Résultats obtenus.....	68
Limites du travail.....	69
Perspectives.....	70
Extension à l'échelle nationale.....	70
Amélioration de la détection.....	70
Enrichissement fonctionnel.....	70
Recherche et publication.....	71
BIBLIOGRAPHIE.....	71
Ouvrages et manuels.....	71
Articles et publications scientifiques.....	72
Documentation technique.....	73
Sources académiques et institutionnelles.....	74

Ressources en ligne.....	75
--------------------------	----

(Cliquez droit sur la table des matières puis « Mettre à jour les champs » pour actualiser la pagination.)

INTRODUCTION GÉNÉRALE

I.1. Préambule

Longtemps considéré comme tabou dans les milieux académiques, le plagiat est aujourd'hui une préoccupation majeure pour la plupart des établissements d'enseignement supérieur à travers le monde. Les universités, jadis réticentes à aborder publiquement la question par crainte de porter atteinte à leur réputation et à la valeur des diplômes qu'elles délivrent, reconnaissent désormais l'ampleur du phénomène et la nécessité d'y apporter une réponse structurée. Cette prise de conscience s'explique par l'évolution des pratiques étudiantes, l'accessibilité croissante des sources numériques et, plus récemment, l'émergence des outils d'intelligence artificielle générative qui bouleversent les modalités traditionnelles de production de texte.

L'avènement d'Internet a amplifié le phénomène plagiaire en offrant aux étudiants un accès quasi illimité à une masse considérable de documents académiques, d'articles scientifiques, d'encyclopédies collaboratives et de travaux antérieurs. Les enquêtes menées dans plusieurs universités africaines et européennes convergent vers un constat identique : près de quatre étudiants sur cinq reconnaissent avoir eu recours au « copier-coller » au moins une fois au cours de leur parcours universitaire. Cette proportion a encore augmenté depuis 2022 avec la diffusion massive des grands modèles de langage tels que ChatGPT, Claude ou Gemini, capables de produire en quelques secondes des textes cohérents, structurés et difficilement détectables par les logiciels anti-plagiat classiques.

Face à cette évolution, les dispositifs institutionnels de lutte contre le plagiat, lorsqu'ils existent, reposent encore largement sur des comparaisons lexicales mot à mot, sur des empreintes numériques (fingerprinting) ou sur des bases de données fermées. Ces approches, bien qu'utiles, montrent leurs limites face aux reformulations, aux paraphrases automatiques et à la traduction multilingue. La détection du plagiat doit donc évoluer pour intégrer les techniques modernes de l'Intelligence Artificielle, notamment le traitement automatique du langage naturel, les embeddings vectoriels et la similarité sémantique.

« Le développement d'une plateforme web basée sur l'Intelligence Artificielle pour la détection automatique du plagiat dans les travaux académiques » constitue le thème du présent mémoire. Il s'inscrit dans une dynamique de modernisation des

dispositifs d'évaluation de l'originalité des travaux scientifiques produits au sein de l'Université de Kinshasa.

La Faculté des Sciences de l'Université de Kinshasa servira de cadre pilote pour la conception et l'expérimentation de la plateforme. Elle constituera un terrain d'expérimentation permettant de valider la solution avant son extension progressive aux autres facultés et, à terme, à d'autres établissements universitaires nationaux. Le présent travail ne se limite donc pas à la réalisation d'un simple logiciel de comparaison de documents : il propose une plateforme web intelligente, modulaire et évolutive, capable d'analyser automatiquement les travaux académiques et de produire des rapports détaillés destinés aux enseignants et aux jurys d'évaluation.

I.2. Problématique

La problématique du plagiat académique ne se résume plus, aujourd'hui, à la simple reproduction non déclarée d'un passage d'un ouvrage ou d'un article. Elle s'est considérablement complexifiée avec l'apparition des technologies numériques, l'accessibilité des bases de données scientifiques en ligne, l'usage massif des moteurs de recherche et, depuis 2022, la prolifération des intelligences artificielles génératives capables de produire du contenu textuel à la demande. Les enseignants et les jurys d'évaluation se trouvent ainsi confrontés à des formes de plagiat de plus en plus sophistiquées, difficiles à identifier par une simple lecture ou par des outils de comparaison superficiels.

À la Faculté des Sciences de l'Université de Kinshasa, le constat est sans appel. L'institution ne dispose d'aucun système informatique dédié à la gestion et au contrôle des travaux de fin de cycle (TFC) et des travaux de fin d'études (TFE). Les travaux sont actuellement déposés sous format papier ou électronique, archivés de manière dispersée, sans indexation systématique et sans mécanisme automatique de comparaison entre les productions étudiantes. Cette situation laisse un large champ libre aux pratiques plagiaires, qu'il s'agisse de la reproduction d'un travail antérieur d'un autre étudiant, du copier-coller massif depuis des sources en ligne, ou désormais, de la génération de contenu par des intelligences artificielles conversationnelles.

Plusieurs questions fondamentales découlent de ce constat. Comment détecter efficacement les similitudes entre un travail soumis et l'ensemble des travaux antérieurs déposés au sein de la faculté ? Comment identifier les paraphrases et les reformulations qui échappent aux comparaisons lexicales classiques ? Comment prendre en compte la

dimension multilingue des sources exploitées par les étudiants, qui consultent aussi bien des documents en français qu'en anglais ? Comment produire un rapport exploitable par l'enseignant, indiquant non seulement un pourcentage global de similarité, mais aussi les passages précis concernés et les sources correspondantes ? Comment, enfin, concevoir une plateforme suffisamment modulaire pour être étendue à d'autres facultés et à d'autres universités ?

Les approches classiques de détection du plagiat, fondées sur la comparaison mot à mot, l'analyse des n-grammes, l'empreinte numérique (fingerprinting) ou la recherche de chaînes de caractères (string matching), présentent des limites bien documentées. Elles ne détectent pas les paraphrases, restent aveugles aux reformulations syntaxiques, ignorent les traductions et ne savent pas reconnaître le contenu produit par les grands modèles de langage. Ces limites rendent nécessaire le recours à des techniques modernes d'Intelligence Artificielle, capables d'analyser non plus seulement la forme des textes, mais également leur sens.

La question centrale qui guide ce travail peut être formulée ainsi : dans quelle mesure le développement d'une plateforme web basée sur les techniques modernes d'Intelligence Artificielle, notamment les embeddings vectoriels et la similarité sémantique, permettrait-il d'améliorer significativement la détection automatique du plagiat dans les travaux académiques à la Faculté des Sciences de l'Université de Kinshasa, tout en posant les bases d'une solution extensible à l'ensemble du système universitaire national ?

I.3. Hypothèse du travail

L'hypothèse principale du présent mémoire est que la conception et le déploiement d'une plateforme web intelligente, fondée sur les techniques modernes d'Intelligence Artificielle et plus particulièrement sur les embeddings vectoriels produits par des modèles de type Sentence-BERT associés à une base de données vectorielle, permettrait de détecter de manière significativement plus efficace les cas de plagiat dans les travaux académiques, par rapport aux approches classiques de comparaison lexicale.

Plus précisément, une telle plateforme serait capable d'identifier non seulement les reproductions strictes (copier-coller), mais aussi les paraphrases, les reformulations syntaxiques et potentiellement les traductions, en s'appuyant sur une représentation vectorielle du sens des phrases plutôt que sur la seule comparaison des chaînes de caractères. L'architecture modulaire de la plateforme permettrait son extension progressive à d'autres

facultés et universités, sans refonte majeure du système, et l'intégration de l'OCR (reconnaissance optique de caractères) garantirait la prise en charge des travaux soumis sous format PDF ou image.

Cette hypothèse sera validée à travers l'expérimentation de la plateforme sur un corpus pilote de travaux académiques de la Faculté des Sciences, en comparant les performances de l'approche sémantique à celles d'une approche lexicale classique, sur la base de métriques d'évaluation standard de l'apprentissage automatique (précision, rappel, F1-score, courbe ROC).

I.4. Choix et intérêt du sujet

Le choix du sujet répond à plusieurs motivations complémentaires. Sur le plan scientifique, il s'agit de contribuer à l'application des techniques modernes d'Intelligence Artificielle, et en particulier du traitement automatique du langage naturel, à un problème concret du monde académique africain. La plupart des recherches publiées dans le domaine de la détection automatique du plagiat portent sur des corpus anglophones et utilisent des modèles entraînés principalement sur des textes en anglais. Le présent travail propose d'évaluer la pertinence de ces approches dans un contexte francophone et africain, où les ressources linguistiques sont plus limitées et où les pratiques académiques présentent des spécificités propres.

Sur le plan pédagogique, la plateforme vise à encourager les étudiants à développer un esprit de créativité, d'innovation et d'originalité dans la production de leurs travaux scientifiques. En disposant d'un outil capable de détecter les similitudes avec les travaux antérieurs, l'étudiant est incité à produire un contenu authentique et à citer correctement ses sources. La plateforme constitue ainsi un dispositif à la fois préventif et curatif : préventif parce qu'elle dissuade les pratiques plagiaires, curatif parce qu'elle les identifie lorsqu'elles se produisent.

Sur le plan institutionnel, l'intérêt est de doter la Faculté des Sciences, puis progressivement l'ensemble de l'Université de Kinshasa, d'un outil numérique moderne de gestion et de contrôle des travaux académiques. Cet outil contribue à la valorisation des diplômes délivrés, à la crédibilité scientifique de l'institution et à la conformité aux standards internationaux en matière d'intégrité académique. La valeur d'un diplôme dépend en grande partie de la confiance accordée à l'originalité des travaux qui ont conduit à sa délivrance.

Sur le plan socio-économique enfin, le projet s'inscrit dans une dynamique de renforcement des capacités numériques locales. Le développement de la plateforme mobilise des compétences en programmation, en apprentissage automatique, en conception de bases de données et en déploiement d'applications web. Il participe à la formation d'une nouvelle génération d'informaticiens congolais capables de concevoir, réaliser et maintenir des systèmes d'information complexes, en réduisant la dépendance technologique extérieure.

I.5. Délimitation du sujet

Pour répondre aux normes scientifiques et assurer la faisabilité du travail dans les délais impartis, le présent mémoire est délimité dans le temps, dans l'espace et dans son contenu.

I.5.1. Délimitation dans le temps

L'étude s'étend sur la période correspondant à l'année académique en cours, durant laquelle ont été conduites les activités de revue de la littérature, d'analyse des besoins, de conception, d'implémentation et d'expérimentation de la plateforme. Le corpus pilote de travaux académiques utilisé pour l'évaluation se compose de mémoires et de TFC effectivement déposés à la Faculté des Sciences au cours des cinq dernières années académiques, afin de disposer d'un volume suffisant pour évaluer les performances du système dans des conditions réelles.

I.5.2. Délimitation dans l'espace

Sur le plan spatial, le travail prend la Faculté des Sciences de l'Université de Kinshasa comme cadre pilote d'expérimentation. Ce choix se justifie par la proximité institutionnelle, l'accessibilité du terrain et la disponibilité d'un corpus documentaire suffisant. Toutefois, la plateforme est conçue dès l'origine comme une solution générique, applicable à d'autres facultés de la même université et extensible à d'autres établissements d'enseignement supérieur nationaux. L'architecture modulaire retenue permet cette extension sans refonte du cœur technique du système.

I.5.3. Délimitation du contenu

Le présent mémoire ne prétend pas traiter l'ensemble des formes de plagiat existantes, ni couvrir toutes les technologies d'Intelligence Artificielle disponibles. Il se concentre sur la détection des similitudes textuelles et sémantiques entre travaux académiques, en s'appuyant

sur les techniques d'embeddings vectoriels et de recherche vectorielle. La détection du plagiat par IA générative, c'est-à-dire l'identification de textes produits par des modèles de langage de type ChatGPT, est abordée comme perspective de recherche, mais ne constitue pas le cœur de l'expérimentation, compte tenu de la maturité encore partielle des solutions existantes dans ce domaine.

I.6. Méthodes et techniques utilisées

La démarche méthodologique adoptée combine plusieurs méthodes et techniques complémentaires, issues à la fois du génie logiciel classique et de la science des données moderne. Cette combinaison se justifie par la double nature du projet : d'une part, la conception d'une plateforme web opérationnelle, qui relève de l'ingénierie logicielle ; d'autre part, l'intégration et l'évaluation d'un modèle d'Intelligence Artificielle, qui relève de la démarche data science.

I.6.1. Méthodes d'analyse et de conception

Pour la modélisation du système d'information, la méthode Merise a été retenue. Elle permet de structurer la conception de la base de données relationnelle en distinguant les niveaux conceptuel, logique et physique, et reste largement enseignée et pratiquée dans les départements d'informatique des universités francophones. La modélisation UML (Unified Modeling Language) est utilisée en complément, pour la représentation des cas d'utilisation, des classes, des séquences et des activités. Ces deux méthodes classiques fournissent un cadre rigoureux pour la spécification de l'architecture logicielle et de la structure des données.

I.6.2. Méthodologie data science : CRISP-DM

Pour la composante Intelligence Artificielle du projet, la méthodologie CRISP-DM (Cross-Industry Standard Process for Data Mining) a été adoptée. Elle structure le projet en six phases : compréhension du domaine, compréhension des données, préparation des données, modélisation, évaluation et déploiement. Cette méthodologie, standard de référence en science des données, garantit une démarche rigoureuse et reproductible, depuis l'analyse du besoin jusqu'à l'évaluation quantitative des performances du modèle.

I.6.3. Techniques de collecte de données

Plusieurs techniques ont été mobilisées pour collecter les informations nécessaires au travail :

- La technique documentaire, qui a permis de consulter des ouvrages, des articles scientifiques, des thèses, des mémoires et des ressources en ligne en rapport avec le plagiat académique, l'Intelligence Artificielle, le traitement automatique du langage naturel et la détection automatique du plagiat.
- La technique d'interview, conduite auprès du doyen de la Faculté des Sciences, des chefs de départements et de plusieurs enseignants, afin de comprendre les pratiques actuelles de dépôt et d'évaluation des travaux académiques, ainsi que les attentes vis-à-vis d'une plateforme anti-plagiat.
- La technique d'observation, qui a consisté à observer directement les procédures de soumission, d'archivage et d'évaluation des travaux au sein de la faculté, ainsi qu'à examiner la structure et le contenu d'un échantillon de mémoires antérieurs.
- La technique d'expérimentation, qui a consisté à constituer un corpus pilote de travaux académiques, à implémenter le pipeline de détection, et à évaluer les performances du système à l'aide de métriques quantitatives.

I.6.4. Techniques d'évaluation

L'évaluation des performances du système s'appuie sur les métriques classiques de l'apprentissage automatique : la précision (proportion des détections effectivement pertinentes), le rappel (proportion des cas réels de plagiat effectivement détectés), le F1-score (moyenne harmonique entre précision et rappel) et la courbe ROC (Receiver Operating Characteristic) avec calcul de l'aire sous la courbe (AUC). Ces métriques permettent de comparer objectivement l'approche sémantique proposée à une approche lexicale classique servant de baseline.

I.7. Subdivision du travail

Hormis l'introduction et la conclusion générales, le présent mémoire est subdivisé en quatre chapitres. Le premier chapitre présente les fondements de l'Intelligence Artificielle appliquée à la détection automatique du plagiat, en couvrant les concepts d'IA, de machine learning, de deep learning, de traitement automatique du langage naturel, d'embeddings

vectoriels, de similarité sémantique et d'IA générative. Le deuxième chapitre propose un état de l'art du plagiat académique, en intégrant les nouvelles formes de plagiat liées à l'IA générative, et analyse les besoins spécifiques du cadre pilote. Le troisième chapitre est consacré à la conception de la plateforme web intelligente, avec une description de l'architecture modulaire, du pipeline IA et des choix de modélisation. Le quatrième chapitre présente l'implémentation, l'expérimentation et l'évaluation de la plateforme, ainsi que l'analyse des résultats obtenus.

CHAPITRE I — FONDEMENTS DE L'INTELLIGENCE ARTIFICIELLE APPLIQUÉE À LA DÉTECTION AUTOMATIQUE DU PLAGIAT

I.1. Introduction

Ce chapitre pose les fondements théoriques et techniques nécessaires à la compréhension du présent mémoire. Il replace d'abord l'Intelligence Artificielle dans son contexte historique et conceptuel, avant d'introduire les principaux paradigmes qui sous-tendent les systèmes modernes de traitement automatique du langage : le machine learning, le deep learning, et plus spécifiquement les modèles fondés sur l'architecture Transformer. Il présente ensuite les notions d'embeddings vectoriels, de similarité cosinus, de recherche vectorielle et d'IA générative, qui constituent le socle technique de la plateforme proposée. Le chapitre se conclut par un état de l'art synthétique des approches modernes de détection automatique du plagiat fondées sur l'IA.

L'objectif n'est pas de produire un traité exhaustif d'Intelligence Artificielle, mais de fournir au lecteur les repères indispensables pour comprendre les choix techniques opérés dans les chapitres suivants. Les notions introduites ici seront mobilisées à la fois dans la conception de la plateforme (chapitre III) et dans son implémentation (chapitre IV).

I.2. Intelligence Artificielle : définition et évolution

L'Intelligence Artificielle (IA) peut être définie comme l'ensemble des techniques visant à doter des systèmes informatiques de capacités cognitives habituellement considérées comme propres à l'intelligence humaine : perception, raisonnement, apprentissage, compréhension du langage, prise de décision. Le terme est introduit formellement en 1956 lors de la conférence de Dartmouth, à l'initiative de John McCarthy, Marvin Minsky, Nathaniel Rochester et Claude Shannon. Depuis lors, le domaine a connu plusieurs cycles d'expansion et de repli, marqués par des hivers successifs suivis de renaissances spectaculaires.

On distingue traditionnellement l'IA faible, qui conçoit des systèmes capables d'accomplir une tâche précise (jouer aux échecs, reconnaître une image, traduire un texte), de l'IA forte, qui viserait à doter une machine d'une compréhension générale et d'une conscience

comparable à celle de l'homme. La quasi-totalité des systèmes opérationnels à ce jour relèvent de l'IA faible, y compris les grands modèles de langage les plus performants. La plateforme proposée dans ce mémoire s'inscrit dans ce cadre : il s'agit d'une IA spécialisée dans la détection de similarités textuelles, et non d'un système doté d'une compréhension générale du monde académique.

L'évolution de l'IA peut être schématiquement divisée en trois grandes vagues. La première vague, des années 1950 aux années 1980, est dominée par les approches symboliques et les systèmes experts, fondés sur la représentation explicite de connaissances sous forme de règles logiques. La deuxième vague, à partir des années 1990, est celle de l'apprentissage statistique, où les systèmes apprennent des régularités à partir de grandes quantités de données. La troisième vague, à partir des années 2010, est celle du deep learning, qui exploite des réseaux de neurones profonds pour apprendre des représentations hiérarchiques complexes. Le présent mémoire s'inscrit explicitement dans cette troisième vague.

Plus récemment, l'année 2022 a marqué un tournant avec l'apparition des grands modèles de langage conversationnels accessibles au grand public, tels que ChatGPT. Ces modèles, fondés sur l'architecture Transformer et entraînés sur des corpus textuels massifs, sont capables de produire des textes cohérents sur des sujets très variés. Leur diffusion pose un défi inédit à l'intégrité académique, car ils rendent possible la génération quasi instantanée de travaux universitaires complets, ce qui impose en retour l'évolution des dispositifs de détection du plagiat.

I.3. Machine Learning

Le Machine Learning, ou apprentissage automatique, est la branche de l'Intelligence Artificielle qui concerne la conception de systèmes capables d'apprendre à partir de données, sans être explicitement programmés pour chaque cas traité. Tom Mitchell propose une définition formelle souvent citée : « un programme informatique apprend d'une expérience E, vis-à-vis d'une tâche T et d'une mesure de performance P, si sa performance sur T, mesurée par P, s'améliore avec l'expérience E ». Cette définition met l'accent sur l'amélioration graduelle des performances par l'exposition aux données, par opposition à la programmation par règles.

On distingue traditionnellement trois grands paradigmes d'apprentissage. L'apprentissage supervisé consiste à entraîner un modèle à partir d'un ensemble de données

étiquetées, où chaque exemple est associé à la sortie attendue. L'apprentissage non supervisé, à l'inverse, travaille sur des données non étiquetées et cherche à en découvrir la structure intrinsèque, par exemple en regroupant les exemples similaires. L'apprentissage par renforcement, enfin, conçoit des agents qui apprennent par essais et erreurs, en recevant des récompenses ou des pénalités pour leurs actions. La détection de plagiat telle qu'abordée dans ce mémoire relève principalement de l'apprentissage non supervisé, les embeddings vectoriels étant calculés sans étiquetage manuel des cas de plagiat.

Dans le contexte de la détection automatique du plagiat, le machine learning intervient à plusieurs niveaux. Il permet tout d'abord de produire des représentations vectorielles des phrases et des paragraphes, qui capturent le sens du texte au-delà de sa forme superficielle. Il permet ensuite de calculer des scores de similarité entre ces représentations, d'identifier les passages potentiellement plagiés et de les classer par ordre de pertinence. Enfin, il autorise l'apprentissage de seuils optimaux de décision, à partir d'un corpus étiqueté de cas avérés de plagiat, afin de minimiser les faux positifs et les faux négatifs.

I.4. Deep Learning

Le Deep Learning, ou apprentissage profond, est une sous-discipline du machine learning qui repose sur l'utilisation de réseaux de neurones artificiels comportant un grand nombre de couches. L'adjectif « profond » fait référence à la profondeur du réseau, c'est-à-dire au nombre de couches de traitement successives entre l'entrée et la sortie. Cette profondeur permet au modèle d'apprendre des représentations hiérarchiques des données, chaque couche transformant la représentation de la couche précédente pour en extraire des informations de plus en plus abstraites.

Un neurone artificiel, ou unité, calcule une combinaison linéaire de ses entrées, à laquelle il applique une fonction d'activation non linéaire (sigmoïde, ReLU, tanh, etc.). L'assemblage de plusieurs neurones en couches successives constitue un réseau de neurones. L'apprentissage consiste à ajuster les poids des connexions entre neurones afin de minimiser une fonction de coût, mesurant l'écart entre les prédictions du modèle et les sorties attendues. Cette optimisation est généralement réalisée par la méthode de rétropropagation du gradient (backpropagation), combinée à un algorithme d'optimisation tel que la descente de gradient stochastique.

Plusieurs architectures de deep learning ont marqué le domaine du traitement automatique du langage naturel. Les réseaux de neurones récurrents (RNN) et leur variante

LSTM (Long Short-Term Memory) ont longtemps dominé le traitement des séquences textuelles, en raison de leur capacité à prendre en compte la dépendance entre les mots successifs. Les réseaux convolutifs (CNN), initialement développés pour la vision par ordinateur, ont également été adaptés au texte. Mais c'est l'architecture Transformer, introduite par Vaswani et al. en 2017, qui a constitué la véritable rupture, en remplaçant la récurrence par un mécanisme d'attention permettant de modéliser directement les relations entre tous les mots d'une séquence, indépendamment de leur position.

I.5. Traitement Automatique du Langage Naturel (NLP)

Le Traitement Automatique du Langage Naturel (TALN), ou Natural Language Processing (NLP) en anglais, est la branche de l'Intelligence Artificielle qui s'intéresse à la compréhension, à l'analyse et à la génération du langage humain par des systèmes informatiques. Le NLP se situe à l'intersection de l'informatique, de la linguistique et de l'apprentissage automatique. Il couvre un large éventail de tâches : traduction automatique, analyse de sentiment, résumé automatique, reconnaissance d'entités nommées, question-réponse, classification de texte, et bien sûr détection de similarité textuelle.

Le traitement d'un texte par un système NLP passe généralement par plusieurs étapes préliminaires. La tokenisation consiste à découper le texte en unités discrètes (tokens), qui peuvent être des mots, des sous-mots ou des caractères selon l'approche retenue. La normalisation regroupe les opérations de mise en minuscules, de suppression de la ponctuation, de suppression des accents ou de traitement des abréviations. La lemmatisation ramène chaque mot à sa forme canonique (le lemme), tandis que la racinisation (stemming) se contente d'en extraire une racine approximative. L'étiquetage morphosyntaxique (POS tagging) associe à chaque token sa catégorie grammaticale. Enfin, la reconnaissance d'entités nommées identifie les personnes, lieux, organisations, dates et autres entités mentionnées dans le texte.

Ces étapes préalables, longtemps considérées comme indispensables, tendent à être intégrées dans les modèles modernes fondés sur l'architecture Transformer. Les modèles comme BERT (Bidirectional Encoder Representations from Transformers), introduit par Devlin et al. en 2018, apprennent des représentations contextuelles des mots, c'est-à-dire que la représentation d'un mot dépend de l'ensemble du contexte dans lequel il apparaît. Cette contextualisation constitue un progrès majeur par rapport aux embeddings statiques comme

Word2Vec ou GloVe, où chaque mot avait une représentation unique quelle que soit la phrase dans laquelle il apparaissait.

Dans le contexte de la détection automatique du plagiat, le NLP intervient à plusieurs étapes du pipeline proposé. Il assure le prétraitement des documents (nettoyage, segmentation, normalisation), la production des embeddings vectoriels des phrases et paragraphes, et l'analyse fine des similarités détectées (par exemple pour distinguer une citation correctement référencée d'un plagiat caractérisé).

I.6. Embeddings vectoriels

Les embeddings vectoriels constituent la pierre angulaire des approches modernes de similarité textuelle. Un embedding est une représentation vectorielle d'une unité linguistique (mot, phrase, paragraphe, document) dans un espace vectoriel de dimension réduite, généralement comprise entre 100 et 1500 dimensions. Cette représentation est apprise de manière à ce que des unités sémantiquement proches correspondent à des vecteurs proches dans l'espace, au sens d'une métrique de distance telle que la distance euclidienne ou la similarité cosinus.

Historiquement, les premiers travaux sur les embeddings ont porté sur la représentation des mots. Le modèle Word2Vec, proposé par Mikolov et al. en 2013, repose sur l'idée que la représentation d'un mot peut être apprise en prédisant les mots qui apparaissent dans son voisinage. GloVe (Global Vectors), proposé par Pennington et al. en 2014, s'appuie sur la matrice de co-occurrence des mots. Ces modèles produisent des embeddings statiques, c'est-à-dire qu'un mot donné est toujours représenté par le même vecteur, quelle que soit la phrase dans laquelle il apparaît. Cette limite est levée par BERT, qui produit des embeddings contextuels.

Pour représenter des unités plus longues que le mot (phrase, paragraphe, document), plusieurs stratégies sont possibles. La plus simple consiste à calculer la moyenne des embeddings des mots qui composent l'unité. Cette approche, rapide, souffre toutefois de limites sémantiques : elle ignore l'ordre des mots et perd une partie de l'information contextuelle. Le modèle Sentence-BERT, introduit par Reimers et Gurevych en 2019, propose une approche plus sophistiquée : il fine-tune BERT spécifiquement pour produire des embeddings de phrases sémantiquement pertinents, en utilisant un mécanisme de pooling sur les représentations de tokens.

Le choix du modèle d'embeddings dépend de plusieurs facteurs : la langue des documents à analyser, la disponibilité de modèles pré-entraînés, les ressources de calcul disponibles, et le compromis entre précision et vitesse. Pour le présent mémoire, le modèle *distiluse-base-multilingual-cased-v1*, version multilingue de Sentence-BERT, a été retenue en raison de sa capacité à traiter le français et l'anglais, de sa taille réduite (adaptée à un déploiement sur des serveurs aux ressources limitées) et de ses performances documentées sur les tâches de similarité sémantique.

Tableau I.1 — Principaux modèles d'embeddings vectoriels utilisables pour la détection de plagiat

Modèle	Année	Type	Dimension	Caractéristique principale
Word2Vec	2013	Statique	300	Prédiction de contexte
GloVe	2014	Statique	100-300	Matrice de co-occurrence
BERT	2018	Contextuel	768-1024	Encodeur bidirectionnel
Sentence-BERT	2019	Phrases	384-768	Pooling sémantique
multilingual SBERT	2019+	Multilingue	512	Cross-lingual

I.7. Similarité cosinus et métriques de distance

Une fois les embeddings vectoriels produits, il faut être en mesure de comparer les vecteurs entre eux pour identifier les similarités. Plusieurs métriques de distance ou de similarité peuvent être utilisées. La plus courante dans le contexte du traitement automatique du langage est la similarité cosinus, qui mesure le cosinus de l'angle entre deux vecteurs dans l'espace vectoriel.

Mathématiquement, la similarité cosinus entre deux vecteurs A et B est définie comme le rapport entre leur produit scalaire et le produit de leurs normes euclidiennes. Elle prend ses valeurs entre -1 (vecteurs opposés) et 1 (vecteurs colinéaires), la valeur 0 correspondant à des vecteurs orthogonaux. Dans le contexte des embeddings de texte, où les vecteurs sont généralement positifs, les valeurs sont comprises entre 0 et 1, une valeur proche de 1 indiquant une forte similarité sémantique.

D'autres métriques peuvent être envisagées : la distance euclidienne, qui mesure la distance géométrique entre deux vecteurs ; la distance de Manhattan, qui somme les différences absolues sur chaque dimension ; la distance de Jaccard, adaptée à la comparaison d'ensembles. Pour les embeddings denses produits par les modèles modernes, la similarité cosinus est généralement privilégiée car elle est invariante par mise à l'échelle, ce qui correspond à l'intuition que deux phrases exprimant la même idée avec des longueurs différentes doivent être considérées comme similaires.

Dans la plateforme proposée, un seuil de similarité est défini pour décider si deux passages doivent être considérés comme potentiellement plagiés. Le choix de ce seuil est un compromis entre précision (éviter les faux positifs) et rappel (ne pas manquer de vrais cas de plagiat). Une valeur typiquement utilisée avec Sentence-BERT est de l'ordre de 0,75 à 0,85, mais ce seuil doit être calibré expérimentalement sur le corpus cible.

I.8. Recherche vectorielle

Lorsque le corpus documentaire à comparer devient important, le calcul exhaustif de la similarité entre un document requête et tous les documents de la base devient prohibitif. La recherche vectorielle, ou recherche approximative des plus proches voisins (Approximate Nearest Neighbors ou ANN), adresse ce problème en organisant les vecteurs dans des structures de données permettant une recherche rapide, au prix d'une légère perte de précision.

Plusieurs bibliothèques implémentent ces techniques. FAISS (Facebook AI Similarity Search), développée par Meta, est l'une des plus utilisées en recherche. Elle propose plusieurs index optimisés (IndexFlatL2, IVF, HNSW, PQ) selon le compromis recherché entre vitesse, mémoire et précision. Milvus est une base de données vectorielle distribuée conçue pour la production. Pinecone et Weaviate sont des services managés. Pour les déploiements plus simples, l'extension pgvector de PostgreSQL permet de stocker et de rechercher des vecteurs directement dans une base relationnelle, sans recourir à un système externe.

Dans le contexte de ce mémoire, l'extension pgvector a été retenue pour plusieurs raisons. D'abord, elle s'intègre nativement à PostgreSQL, qui est déjà utilisé pour les données relationnelles de la plateforme (utilisateurs, travaux, analyses), ce qui évite l'introduction d'un système supplémentaire. Ensuite, elle supporte les index HNSW (Hierarchical Navigable Small World) qui offrent d'excellentes performances de recherche. Enfin, elle

permet d'utiliser le langage SQL pour exprimer des requêtes combinant critères relationnels et similarité vectorielle, ce qui simplifie l'implémentation.

I.9. IA générative et grands modèles de langage

L'IA générative désigne la branche de l'Intelligence Artificielle qui vise à produire du contenu nouveau (texte, image, audio, vidéo) à partir de modèles appris sur de grandes quantités de données. Dans le domaine textuel, cette IA générative s'appuie sur les grands modèles de langage (Large Language Models ou LLM), dont les plus connus sont la famille GPT (Generative Pre-trained Transformer) d'OpenAI, Claude d'Anthropic, Gemini de Google, et les modèles ouverts comme Llama (Meta), Mistral ou DeepSeek.

Un grand modèle de langage est un modèle neuronal de type Transformer, entraîné sur un corpus textuel massif (plusieurs milliards voire trillions de tokens) à prédire le token suivant dans une séquence. Cette tâche apparemment simple, lorsqu'elle est réalisée à l'échelle, conduit le modèle à internaliser une représentation riche de la syntaxe, de la sémantique et même d'une partie des connaissances factuelles présentes dans les données d'entraînement. Le modèle peut ensuite être spécialisé pour diverses tâches par affinage (fine-tuning) ou par adaptation en contexte (in-context learning).

L'irruption de l'IA générative dans le champ académique pose un défi inédit à la détection du plagiat. Contrairement au plagiat classique, qui consiste à recopier un texte existant, le plagiat par IA générative produit un texte nouveau, qui n'existe dans aucune source antérieure identifiable. Les outils classiques de détection de similarité sont donc impuissants. Plusieurs approches émergent pour tenter de détecter ce type de plagiat : analyse statistique de la perplexité du texte (un texte généré par IA tend à avoir une perplexité plus faible qu'un texte humain), analyse stylométrique (les LLM ont des tics d'écriture reconnaissables), ou utilisation de classificateurs entraînés à distinguer texte humain et texte généré. Aucune de ces approches n'est à ce jour pleinement satisfaisante, et la détection du plagiat par IA générative constitue un champ de recherche actif, abordé dans ce mémoire comme perspective.

Il convient également de noter que l'IA générative peut être utilisée comme outil de paraphrase automatique, transformant un texte source en un texte différent mais sémantiquement équivalent. Cette utilisation rend plus difficile la détection par les approches lexicales, mais reste partiellement identifiable par les approches sémantiques basées sur les

embeddings, qui capturent le sens plutôt que la forme. C'est l'un des arguments en faveur de l'approche retenue dans ce mémoire.

I.10. État de l'art de la détection automatique du plagiat par IA

La détection automatique du plagiat a connu plusieurs générations d'approches. La première génération, dominée par les méthodes de string matching et de fingerprinting, repose sur la comparaison lexicale des documents. L'algorithme de Rabin-Karp, les n-grammes de caractères ou de mots, et les empreintes de Winnowing en sont des exemples caractéristiques. Ces approches sont rapides et efficaces pour la détection des copier-coller stricts, mais incapables d'identifier les paraphrases et les reformulations.

La deuxième génération, à partir des années 2010, introduit des méthodes d'apprentissage statistique. Les modèles de topic modeling (LDA), les embeddings de mots (Word2Vec, GloVe) et les modèles de langue (BERT) commencent à être utilisés pour calculer des similarités sémantiques entre documents. Plusieurs travaux académiques (Potthast et al., Stein et al.) proposent des évaluations systématiques de ces approches, notamment dans le cadre des campagnes PAN (Plagiarism Detection, Author Identification, and Sexual Predator Identification).

La troisième génération, émergente, s'appuie sur les embeddings de phrases et les modèles Transformer. Sentence-BERT, SimCSE, et plus récemment les embeddings produits par les LLM (text-embedding-ada-002 d'OpenAI, Cohere Embed, etc.) permettent de calculer des similarités sémantiques fines, même entre des phrases reformulées. Ces approches sont aujourd'hui l'état de l'art pour la détection de similarité intra-corpus, mais leur application à grande échelle reste coûteuse en ressources de calcul et en stockage vectoriel.

La quatrième génération, encore balbutiante, tente de détecter le plagiat par IA générative. Les approches combinent classificateurs supervisés, métriques de perplexité, analyse stylométrique et détection de motifs syntaxiques récurrents. Aucune solution mature n'est à ce jour disponible, et ce défi est explicitement identifié comme l'une des perspectives de recherche du présent mémoire.

Tableau I.2 — Évolution des approches de détection automatique du plagiat

Génération	Approche dominante	Capacités	Limites
1re (années 1990+)	String matching, fingerprinting	Copier-coller strict	Aveugle aux paraphrases

Génération	Approche dominante	Capacités	Limites
2e (années 2010+)	ML statistique, Word2Vec	Similarité lexicale étendue	Pas de contexte
3e (années 2019+)	Sentence-BERT, Transformers	Similarité sémantique	Coût de calcul
4e (émergente)	LLM, classificateurs IA	Détection de texte IA	Fiabilité partielle

I.11. Conclusion

Ce chapitre a posé les fondements conceptuels et techniques du mémoire. L'Intelligence Artificielle, de ses origines symboliques aux architectures Transformer contemporaines, fournit aujourd'hui des outils puissants pour le traitement automatique du langage naturel. Les embeddings vectoriels, et en particulier les embeddings de phrases produits par des modèles comme Sentence-BERT, permettent de capturer le sens des textes au-delà de leur forme superficielle. La similarité cosinus, combinée à des techniques de recherche vectorielle approchée, offre une solution efficace pour identifier les similitudes sémantiques entre travaux académiques, à une échelle compatible avec un déploiement opérationnel.

L'émergence récente de l'IA générative et des grands modèles de langage bouleverse en outre le paysage du plagiat académique, en rendant possible la génération automatique de textes difficiles à détecter par les outils classiques. Cette évolution impose le recours aux approches sémantiques modernes et ouvre de nouvelles perspectives de recherche pour identifier les productions automatisées. Le chapitre suivant examinera en détail l'état de l'art du plagiat académique, ses formes, ses causes et ses conséquences, ainsi que les limites des solutions actuellement disponibles sur le marché.

CHAPITRE II — ÉTAT DE L'ART DU PLAGIAT ACADÉMIQUE ET ANALYSE DES BESOINS

II.1. Introduction

Le plagiat dans le contexte académique est devenu, depuis une trentaine d'années, une préoccupation majeure pour une proportion grandissante d'établissements d'enseignement supérieur à travers le monde. D'abord largement sous-estimé, le phénomène a vu son importance et l'étude de ses tenants et aboutissants connaître un gain d'intérêt constant, sous l'effet conjugué de la massification de l'enseignement supérieur, de la numérisation des sources documentaires et, plus récemment, de l'apparition des intelligences artificielles génératives. L'avènement des technologies de l'information et de la communication, Internet en tête, suscite encore aujourd'hui un vif débat sur les opportunités qu'elles offrent en matière de recherche, mais aussi en termes de pratiques frauduleuses.

De façon récurrente depuis l'intégration d'Internet dans les institutions académiques, des recherches diffusent en effet l'idée que les possibilités de manipulation de l'information offertes par cette technologie favorisent l'émergence de comportements plagiaires parmi les étudiants. De l'aveu de ceux-ci, ces facilités représentent des causes de plagiat modérément à très pertinentes. Il est donc légitime de s'interroger sur les mécanismes de cette relation et sur les implications d'une augmentation du phénomène, ainsi que sur les réponses techniques et institutionnelles à y apporter.

Le présent chapitre propose un état de l'art du plagiat académique, en mobilisant la littérature scientifique existante et en l'enrichissant pour prendre en compte les nouvelles formes de plagiat apparues avec l'IA générative. Il aborde successivement les définitions du plagiat, ses domaines d'application, ses différents types, les méthodes de détection automatique, les causes et les conséquences du phénomène, ainsi que les mesures de lutte existantes. Il se conclut par une analyse comparative des principaux logiciels anti-plagiat disponibles et par une identification des besoins spécifiques du cadre pilote.

II.2. Définition du plagiat

Le plagiat est une faute d'ordre moral, civil ou commercial, qui peut être sanctionnée au pénal. Il consiste à copier un auteur ou à accaparer l'œuvre d'un créateur dans le domaine des

arts sans le citer ou le dire, ainsi qu'à s'inspirer fortement d'un modèle que l'on omet, délibérément ou par négligence, de désigner. Il est souvent assimilé à un vol immatériel, dans la mesure où l'on s'approprie le fruit du travail intellectuel d'autrui pour le présenter comme sien. Le plagiaire est celui qui s'approprie frauduleusement tout ou partie d'une œuvre littéraire, technique, artistique ou scientifique.

Dans le contexte académique, le plagiat recouvre plus largement toute pratique par laquelle un étudiant ou un chercheur présente comme personnel un travail, une idée, une formulation ou un résultat qui provient en réalité d'une source extérieure non citée. Cette définition englobe non seulement la reproduction littérale non référencée, mais aussi la paraphrase non attribuée, la traduction non sourcée, la réutilisation de données sans crédit, l'auto-plagiat (réutilisation de ses propres travaux sans le mentionner) et, depuis peu, l'utilisation non déclarée de contenus produits par des intelligences artificielles génératives.

La frontière entre le plagiat et les pratiques légitimes n'est pas toujours évidente à tracer. Citer un auteur sans reprendre exactement ses mots, sans indiquer la source exacte ou en modifiant légèrement la formulation peut constituer un plagiat selon les standards académiques, même si l'intention de tromper n'est pas présente. C'est cette porosité de la notion qui rend nécessaire l'existence d'outils techniques d'aide à la détection, complémentaires du jugement humain de l'enseignant ou du jury.

II.3. Quelques domaines du plagiat

Le plagiat n'est pas l'apanage du monde académique. Il se rencontre dans de nombreux domaines de l'activité humaine, avec des spécificités propres à chacun.

II.3.1. Dans les sciences

Le plagiat scientifique touche les articles de recherche, les communications, les thèses et les rapports techniques. Il peut prendre la forme de la reproduction de résultats expérimentaux, de l'appropriation d'idées ou de méthodologies, ou de la copie de passages entiers d'articles antérieurs. Les conséquences sont particulièrement lourdes dans ce domaine, car elles affectent la crédibilité de la recherche et peuvent conduire à des rétractations d'articles, à des sanctions professionnelles et à la mise en cause de la validité de travaux ultérieurs qui s'appuient sur les résultats plagiés.

II.3.2. Dans le journalisme

Le plagiat journalistique consiste à reprendre, sans citation, des informations, des enquêtes ou des récits produits par d'autres médias ou d'autres journalistes. Plusieurs affaires médiatiques retentissantes, dans la presse écrite comme à la télévision, ont mis en lumière l'ampleur du phénomène et conduit à des licenciements, des procès et à la perte de crédibilité des organes de presse concernés. La pression de l'immédiateté, la concurrence entre titres et la facilité de l'accès aux sources en ligne constituent des facteurs aggravants.

II.3.3. Dans la littérature et les arts

Le plagiat littéraire et artistique concerne l'appropriation d'œuvres de fiction, de scénarios, de partitions musicales, d'œuvres graphiques ou plastiques. Plusieurs procès célèbres ont opposé des écrivains, des réalisateurs ou des musiciens autour d'accusations de plagiat. La distinction entre influence légitime, hommage et plagiat délibéré y est souvent délicate à établir, et relève autant du droit d'auteur que de la déontologie artistique.

II.3.4. Dans le monde académique

Le plagiat académique, qui constitue le cœur du présent mémoire, concerne les travaux produits dans le cadre d'un cursus d'enseignement : devoirs, dissertations, exposés, travaux de fin de cycle, mémoires, thèses. Il est probablement la forme la plus répandue de plagiat, en raison du nombre d'étudiants concernés, de la pression scolaire et de l'accessibilité des sources. Il affecte l'ensemble des disciplines, des sciences humaines aux sciences dures, et tous les niveaux d'étude, du premier cycle au doctorat.

II.4. Types de plagiat

La littérature scientifique identifie plusieurs types de plagiat, qui diffèrent par leur degré de gravité, leur mode opératoire et leur facilité de détection. Une typologie rigoureuse est importante pour concevoir des outils de détection adaptés et pour évaluer la gravité des cas identifiés.

Tableau II.1 — Typologie des principaux types de plagiat académique

Type de plagiat	Description	Détection
Copier-coller strict	Reproduction littérale sans citation	Facile (méthodes lexicales)

Type de plagiat	Description	Détection
Copier-coller avec références	Reproduction littérale avec citation, mais sans guillemets	Moyenne
Paraphrase non référencée	Reformulation d'un passage sans en citer la source	Difficile (sémantique)
Paraphrase patchwork	Assemblage de paraphrases issues de plusieurs sources	Difficile
Traduction non sourcée	Traduction d'un texte étranger sans en citer la source	Très difficile
Auto-plagiat	Réutilisation de ses propres travaux antérieurs sans le mentionner	Possible par recoupement
Plagiat par IA générative	Production d'un texte par un LLM présenté comme œuvre personnelle	Très difficile (émergent)
Plagiat d'idées	Appropriation d'une idée, d'un concept ou d'une méthodologie	Impossible par automate

Cette typologie met en évidence l'insuffisance des approches purement lexicales, qui ne détectent efficacement que les deux premiers types. Les approches sémantiques fondées sur les embeddings permettent d'élargir la détection aux paraphrases et aux patchworks, ainsi qu'aux traductions lorsque le modèle d'embeddings est multilingue. Le plagiat par IA générative et le plagiat d'idées restent en revanche des défis partiellement ouverts, qui nécessitent des approches complémentaires.

II.5. Détection automatique du plagiat

La détection automatique du plagiat peut être classée en trois catégories principales, selon l'approche technique retenue.

La première catégorie tente de détecter le style d'écriture et d'identifier les changements incompatibles avec ce style. Elle s'appuie sur des analyses stylométriques (longueur des phrases, richesse du vocabulaire, fréquence de certains mots-outils) et suppose que le plagiat introduit des ruptures stylistiques détectables. Cette approche est intéressante mais peu fiable en pratique, notamment pour les travaux étudiants dont le style n'est pas encore stabilisé.

La deuxième catégorie est la plus utilisée. Elle se base sur la comparaison entre plusieurs documents et l'identification des chevauchements de parties entre ces documents.

Plusieurs techniques sont mobilisées : le fingerprinting, qui représente chaque document comme un ensemble d'empreintes de sous-chaînes (n-grammes) et compare ces empreintes ; le string matching, qui compare les documents mot par mot en utilisant des algorithmes comme Brute Force ou Rabin-Karp ; l'analyse des citations, qui examine les séquences de références bibliographiques partagées entre documents. Ces techniques sont efficaces pour la détection du copier-coller strict, mais aveugles aux reformulations.

La troisième catégorie reçoit un document en entrée et recherche les motifs de plagiat dans le Web, de manière manuelle ou automatisée. Elle est utilisée par des services commerciaux comme Turnitin ou Compilatio, qui disposent d'index massifs de pages web et de travaux antérieurs. Cette approche nécessite une connexion Internet, un abonnement souvent coûteux, et soulève des questions de confidentialité lorsqu'on confie à un tiers les travaux des étudiants.

Les approches modernes, fondées sur l'IA et les embeddings vectoriels, complètent ces catégories traditionnelles. Elles ne se limitent pas à la comparaison lexicale, mais mesurent la similarité sémantique entre les passages des documents. Cette approche, retenue dans le présent mémoire, permet de détecter les paraphrases et les reformulations qui échappent aux outils classiques. Elle peut être combinée avec les approches traditionnelles pour maximiser la couverture des différents types de plagiat.

II.6. Causes et facteurs du plagiat académique

Plusieurs études convergent pour identifier un ensemble de facteurs qui favorisent le plagiat en milieu académique. Ces facteurs sont à la fois individuels, institutionnels et technologiques, et leur combinaison explique l'ampleur et la persistance du phénomène.

II.6.1. Facteurs individuels

Au niveau individuel, plusieurs motivations sont fréquemment citées : la pression de la performance scolaire, la peur de l'échec, le manque de temps lié à la conciliation des études avec une activité professionnelle, la difficulté du sujet, le manque de confiance en ses capacités de rédaction, et parfois simplement l'opportunité perçue. Les étudiants les plus en difficulté sont souvent les plus tentés par le plagiat, mais le phénomène touche également des étudiants brillants, en quête d'optimisation de leur temps ou de leur note.

II.6.2. Facteurs institutionnels

Au niveau institutionnel, plusieurs éléments favorisent le plagiat : l'absence de formation méthodologique à la citation et à la rédaction scientifique, l'insuffisance des sanctions en cas de plagiat avéré, l'absence d'outils techniques de détection, le caractère impersonnel des cours en amphithéâtre qui facilite l'anonymat des étudiants, et parfois l'inadéquation entre les exigences académiques et les moyens accordés aux étudiants pour y répondre. La culture institutionnelle, notamment la tolérance plus ou moins grande vis-à-vis du copier-coller, joue également un rôle déterminant.

II.6.3. Facteurs technologiques

Au niveau technologique, l'accessibilité des sources numériques (articles en ligne, encyclopédies collaboratives, blogs, dépôts de mémoires accessibles), la facilité du copier-coller, l'existence d'outils de paraphrase automatique, et désormais la disponibilité gratuite des grands modèles de langage conversationnels, constituent autant de facilités technologiques. Les enquêtes menées auprès des étudiants estiment que quatre étudiants sur cinq ont déjà eu recours au copier-coller, et que la proportion a significativement augmenté depuis la diffusion massive de ChatGPT fin 2022.

II.7. Conséquences du plagiat

Les conséquences du plagiat académique sont multiples et concernent à la fois l'étudiant, l'institution et la société dans son ensemble.

Pour l'étudiant, le plagiat, lorsqu'il est détecté, expose à des sanctions pouvant aller de l'annulation de l'épreuve à l'exclusion définitive de l'établissement. Au-delà des sanctions formelles, le plagiat non détecté conduit à une dégradation des apprentissages : en contournant l'effort de production personnelle, l'étudiant se prive de l'acquisition de compétences essentielles (recherche, analyse, synthèse, rédaction, esprit critique). À terme, des diplômes obtenus partiellement par plagiat peuvent conduire à des incompétences professionnelles aux conséquences graves, notamment dans les métiers à responsabilité.

Pour l'institution, le plagiat non maîtrisé entraîne une dévalorisation des diplômes délivrés et une perte de crédibilité scientifique. Les employeurs et les institutions partenaires, en constatant l'incapacité des diplômés à produire un travail original, peuvent perdre confiance dans la valeur des certifications. Les universités engagées dans des partenariats

internationaux ou dans des démarches d'accréditation sont particulièrement vulnérables sur ce point, l'intégrité académique étant un critère central d'évaluation.

Pour la société, le plagiat académique contribue à une dégradation générale de la confiance dans les productions intellectuelles et scientifiques. Lorsque les travaux de recherche, les rapports techniques ou les publications contiennent des éléments plagiés, la validité des connaissances produites est compromise. Dans les domaines sensibles (santé, ingénierie, droit), les conséquences peuvent être directes et parfois dramatiques. La lutte contre le plagiat s'inscrit donc dans une politique plus large de préservation de l'intégrité intellectuelle.

II.8. Mesures de lutte contre le plagiat

La lutte contre le plagiat académique mobilise plusieurs leviers complémentaires, qui doivent être articulés pour une efficacité maximale. Aucune mesure isolée n'est suffisante : c'est la combinaison d'actions pédagogiques, institutionnelles et techniques qui produit des résultats durables.

II.8.1. Mesures pédagogiques

Sur le plan pédagogique, plusieurs actions sont recommandées : la formation des étudiants aux règles de la citation et de la rédaction scientifique dès le début du cursus, l'intégration de modules sur l'intégrité académique dans les enseignements transversaux, l'accompagnement personnalisé des étudiants en difficulté par des séances de méthodologie, et la conception de sujets de travaux qui réduisent les opportunités de plagiat (sujets originaux, contextualisés, exigeant une réflexion personnelle). Les formations méthodologiques menées seules, sans contrôle effectif, ne font que faiblement baisser le recours au copier-coller : elles doivent donc être combinées à des dispositifs de détection.

II.8.2. Mesures institutionnelles

Sur le plan institutionnel, il convient de mettre en place une politique d'intégrité académique explicite, formalisée dans un règlement intérieur et portée par les autorités de l'établissement. Cette politique doit définir clairement ce qui constitue une faute, les sanctions applicables, la procédure de traitement des cas avérés, et les droits de la défense. Un engagement signé par chaque étudiant en début d'année, attestant de la compréhension

des règles d'intégrité, peut constituer un dispositif complémentaire utile. La transparence des sanctions (sans nécessairement nommer les contrevenants) participe à la prévention.

II.8.3. Mesures techniques

Sur le plan technique, le déploiement d'outils de détection automatique du plagiat constitue un complément indispensable. Ces outils permettent d'une part de dissuader les pratiques plagiaires (effet préventif lié à la connaissance de l'existence du contrôle), d'autre part d'identifier les cas effectifs pour traitement pédagogique ou sanctionnaire. Plusieurs solutions commerciales existent (Turnitin, Compilatio, Urkund), mais leur coût, leur dépendance à des bases de données externes et les questions de confidentialité posées par l'envoi des travaux étudiants à des serveurs étrangers limitent leur adoption dans le contexte africain. Le développement d'une solution locale, comme celle proposée dans ce mémoire, répond à ces enjeux.

II.9. Plagiat assisté par IA générative

Depuis la mise à disposition publique des grands modèles de langage conversationnels à partir de fin 2022, une nouvelle forme de plagiat est apparue dans le monde académique : le plagiat assisté par IA générative. Cette forme de plagiat consiste pour l'étudiant à faire produire tout ou partie de son travail par un modèle d'IA tel que ChatGPT, Claude, Gemini ou un outil similaire, puis à le présenter comme œuvre personnelle.

Cette pratique pose des défis inédits aux dispositifs de détection. Contrairement au plagiat classique, le texte produit par l'IA n'existe dans aucune source antérieure identifiable : il est généré à la demande, en fonction d'une consigne. Les outils de comparaison de documents, même les plus avancés, sont donc impuissants à le détecter par leur mécanisme habituel. Plusieurs approches émergent pour tenter d'identifier ces productions : analyse de la perplexité du texte (un texte généré par IA présente une perplexité moyenne plus faible, car les LLM produisent statistiquement les tokens les plus probables), analyse stylométrique (certains motifs lexicaux et syntaxiques sont caractéristiques des LLM), utilisation de classificateurs supervisés entraînés sur des corpus de textes humains et de textes IA. Aucune de ces approches n'est à ce jour pleinement satisfaisante, avec des taux de faux positifs et de faux négatifs significatifs, et des risques d'accusation injustifiée d'étudiants ayant produit un travail authentique.

L'évolution rapide des LLM (GPT-3.5, GPT-4, GPT-4o, Claude 3, Gemini 1.5 Pro, etc.) rend la détection particulièrement difficile : chaque génération de modèles produit des textes de plus en plus proches des productions humaines, et les classificateurs entraînés sur les générations antérieures deviennent rapidement obsolètes. Plusieurs universités ont d'ailleurs renoncé à utiliser des détecteurs de texte IA, estimant que leur fiabilité était insuffisante et que leur usage pouvait conduire à des accusations infondées. La détection du plagiat par IA générative constitue ainsi l'un des fronts de recherche les plus actifs du domaine, et fait l'objet d'une attention particulière dans les perspectives de ce mémoire.

II.10. Techniques modernes de paraphrase automatique

La paraphrase automatique désigne l'ensemble des techniques permettant de transformer un texte source en un texte cible sémantiquement équivalent mais formellement différent. Ces techniques, longtemps rudimentaires (substitution de synonymes, restructuration syntaxique simple), ont été considérablement améliorées par les modèles de langage récents. Un LLM comme GPT-4 est aujourd'hui capable de produire en quelques secondes une paraphrase fluide et grammaticalement correcte d'un paragraphe donné, modifiant le vocabulaire, la structure des phrases et l'ordre des idées tout en préservant le sens.

Cette capacité de paraphrase automatique constitue un défi direct pour les outils de détection du plagiat fondés sur la comparaison lexicale. Un étudiant peut, à partir d'un texte source non cité, produire une version paraphrasée qui passera inaperçue aux outils classiques, tout en étant sémantiquement équivalente à l'original. Les approches sémantiques basées sur les embeddings, comme celle proposée dans ce mémoire, sont mieux armées pour identifier ce type de plagiat, dans la mesure où elles comparent le sens des passages plutôt que leur forme. Toutefois, les paraphrases très sophistiquées produites par les LLM récents peuvent même tromper les modèles d'embeddings, ce qui souligne la nécessité de combiner plusieurs approches.

Les techniques de paraphrase automatique peuvent également être utilisées de manière positive, comme outil pédagogique pour aider les étudiants à améliorer leur rédaction ou à comprendre les mécanismes de la reformulation. Cette double nature (outil de fraude potentielle et outil d'apprentissage) illustre la complexité du paysage académique contemporain, où les mêmes technologies peuvent être mises au service de l'intégrité ou de la tricherie.

II.11. Limites des logiciels classiques

Les logiciels classiques de détection du plagiat, bien qu'utiles, présentent plusieurs limites bien documentées. La première limite est leur incapacité à détecter les paraphrases et les reformulations syntaxiques : en comparant des chaînes de mots, ils ne voient pas les passages dont le sens est identique mais la formulation différente. La deuxième limite est leur insensibilité aux traductions : un passage traduit d'une langue étrangère sans citation de la source originale échappera à toute détection, à moins que l'outil ne dispose d'un mécanisme de comparaison multilingue.

La troisième limite est l'incapacité à détecter le plagiat par IA générative, dans la mesure où le texte produit n'existe dans aucune source antérieure. Les logiciels classiques ne voient rien, puisqu'ils ne trouvent aucune correspondance dans leur base. Cette limite est devenue critique depuis 2022 et rend nécessaires les évolutions vers des approches complémentaires.

La quatrième limite concerne les questions de confidentialité et de souveraineté des données. Les principaux logiciels commerciaux (Turnitin, Compilatio) fonctionnent en envoyant le document à vérifier sur des serveurs externes, généralement situés à l'étranger. Cette pratique pose problème dans plusieurs contextes : confidentialité des travaux étudiants, propriété intellectuelle des institutions, conformité avec les réglementations locales sur la protection des données. Dans le contexte africain, s'ajoutent la dépendance technologique, le coût des licences en devises fortes, et l'absence de prise en compte des spécificités linguistiques et disciplinaires locales.

Enfin, les logiciels classiques ne distinguent pas les signes diacritiques comme les guillemets, les chevrons, l'italique ou le gras. Ils ne peuvent détecter l'emprunt d'idées, ni juger du sens d'une rédaction. Les paraphrases et autres bricolages syntaxiques échappent à la majorité d'entre eux, tout comme les traductions effectuées à l'aide de logiciels. C'est cette accumulation de limites qui justifie le développement d'approches nouvelles, fondées sur l'IA et la similarité sémantique.

II.12. Comparaison avec les plateformes modernes

Plusieurs plateformes modernes ont fait évoluer leurs approches pour intégrer les techniques d'IA. Le tableau suivant synthétise une comparaison entre solutions commerciales existantes et l'approche proposée dans ce mémoire.

Tableau II.2 — Comparaison entre plateformes anti-plagiat existantes et solution proposée

Plateforme	Approche	Multilingue	Confidentialité	Coût	Adaptation locale
Turnitin	Lexicale + web	Limitée	Données envoyées à l'étranger	Élevé (devise forte)	Faible
Compilatio	Lexicale + IA	Français/anglais	Données envoyées à l'étranger	Élevé	Moyenne
Urkund	Lexicale + web	Plusieurs langues	Données envoyées à l'étranger	Élevé	Faible
Plateforme proposée	Sémantique (embeddings)	Multilingue (SBERT)	Données hébergées localement	Maîtrisé	Forte (corpus local)

Cette comparaison met en évidence les avantages différenciants de la solution proposée : adaptation au contexte linguistique et disciplinaire local, hébergement des données sur l'infrastructure nationale, maîtrise du coût, et capacité à évoluer vers une détection interuniversitaire grâce à son architecture modulaire. La plateforme ne prétend pas concurrencer les solutions commerciales sur la taille de leur index web, mais offre une alternative pertinente pour le contexte universitaire congolais et africain, en s'appuyant sur un corpus local de travaux académiques et sur les techniques modernes de similarité sémantique.

II.13. Conclusion

Ce chapitre a présenté un état de l'art du plagiat académique, en intégrant les évolutions récentes liées à l'IA générative. Le plagiat, dans ses formes classiques (copier-coller, paraphrase, traduction, auto-plagiat) comme dans ses formes émergentes (production par LLM), constitue un défi majeur pour l'intégrité académique. Les outils classiques de détection, fondés sur la comparaison lexicale, présentent des limites qui rendent nécessaires les approches sémantiques modernes. La plateforme proposée dans ce mémoire s'inscrit dans cette dynamique, en combinant embeddings vectoriels, similarité cosinus et architecture modulaire, tout en répondant aux spécificités du contexte local.

Le chapitre suivant sera consacré à la conception de la plateforme : architecture générale, modularité, pipeline IA, modélisation des données et interface fonctionnelle. Il fera le lien entre les fondements théoriques posés dans le chapitre I, l'état de l'art dressé dans le présent chapitre, et l'implémentation qui fera l'objet du chapitre IV.

CHAPITRE III — CONCEPTION D'UNE PLATEFORME WEB INTELLIGENTE BASÉE SUR L'IA

III.1. Introduction

Ce chapitre est consacré à la conception de la plateforme web intelligente objet du présent mémoire. Il opère la transition entre les fondements théoriques posés au chapitre I et l'état de l'art dressé au chapitre II d'une part, et l'implémentation concrète qui fera l'objet du chapitre IV d'autre part. Il présente successivement le cadre pilote d'expérimentation, l'analyse des besoins fonctionnels et non fonctionnels, l'architecture générale et modulaire de la plateforme, le pipeline de traitement IA, la modélisation des données relationnelles et vectorielles, ainsi que les principaux diagrammes UML et schémas d'architecture.

La conception obéit à trois exigences majeures. Premièrement, la plateforme doit être intelligente, c'est-à-dire s'appuyer sur les techniques modernes d'IA décrites au chapitre I pour produire des analyses sémantiques pertinentes. Deuxièmement, elle doit être modulaire, afin de permettre son extension progressive à d'autres facultés, départements et universités sans refonte majeure. Troisièmement, elle doit être opérationnelle, c'est-à-dire déployable sur une infrastructure réaliste dans le contexte congolais, en tenant compte des contraintes de connectivité, de ressources matérielles et de compétences disponibles.

III.2. Présentation du cadre pilote : la Faculté des Sciences

La Faculté des Sciences de l'Université de Kinshasa constitue le cadre pilote d'expérimentation de la plateforme. Sa création remonte à celle de l'Université de Lovanium, devenue Université de Kinshasa, en 1951. La pose de la première pierre du bâtiment de cette faculté, premier bâtiment de l'Université, a eu lieu le 26 septembre 1954. L'ouverture officielle de la Faculté des Sciences a eu lieu en 1957 avec des cycles d'études variés : sciences mathématiques, physiques, chimiques, biologiques, géologiques et minéralogiques. Au fil des décennies, la faculté a élargi son offre de formation pour inclure les études d'ingénieur civil et agronome, la candidature en pharmacie, les licences en sciences et un certificat en météorologie.

Aujourd'hui, la Faculté des Sciences comporte cinq départements : biologie, chimie, mathématiques et informatique, physique, et sciences de la terre. Elle organise une

propédeutique en physique-mathématique, un diplôme spécial en gestion de l'environnement, et dispose d'un centre de formation en informatique. Elle mène, malgré des moyens matériels limités, des études tant fondamentales qu'appliquées, contribuant ainsi au développement de la science et à l'amélioration de l'environnement. Pour l'année académique en cours, la faculté compte une soixantaine de professeurs, plus de trente chefs de travaux, une soixantaine d'assistants et chargés de pratiques professionnelles, et environ quatre-vingts agents administratifs. Sur les dix facultés que renferme l'Université de Kinshasa, la Faculté des Sciences assure également les enseignements dans neuf autres facultés.

Le choix de la Faculté des Sciences comme cadre pilote se justifie par plusieurs facteurs : la disponibilité d'un corps professoral sensibilisé aux enjeux technologiques, l'existence d'un département d'informatique disposant de compétences techniques mobilisables, l'accessibilité d'un corpus documentaire suffisant (mémoires et TFC antérieurs), et la proximité institutionnelle facilitant l'expérimentation. Toutefois, ce choix ne limite en rien la portée de la solution : la plateforme est conçue dès l'origine comme une solution générique, applicable à toute faculté ou université, et le cadre pilote sert uniquement à valider la pertinence de l'approche dans des conditions réelles.

III.3. Analyse des besoins fonctionnels

L'analyse des besoins fonctionnels a été conduite à partir des interviews menées auprès du doyen de la Faculté des Sciences, des chefs de départements et de plusieurs enseignants, complétées par l'observation directe des procédures actuelles de dépôt et d'évaluation des travaux. Elle a permis d'identifier l'ensemble des fonctionnalités que la plateforme doit offrir pour répondre aux besoins exprimés.

III.3.1. Gestion des utilisateurs et authentification

La plateforme doit gérer plusieurs types d'utilisateurs avec des rôles et des permissions distincts. Les étudiants constituent le premier type d'utilisateurs : ils peuvent s'authentifier, déposer leurs travaux (TFC ou mémoires), consulter l'historique de leurs dépôts et accéder aux rapports d'analyse qui les concernent. Les enseignants constituent le second type : ils peuvent s'authentifier, accéder aux travaux déposés par les étudiants qu'ils encadrent, consulter les rapports d'analyse, valider ou rejeter les travaux, et laisser des commentaires. Les administrateurs de faculté constituent le troisième type : ils gèrent les comptes utilisateurs, les départements, les promotions, et disposent d'un tableau de bord global. Enfin,

un super-administrateur a la responsabilité de l'ensemble de la plateforme, de la gestion des facultés et des paramètres techniques.

III.3.2. Gestion académique

La plateforme doit permettre la gestion structurée de l'organisation académique : facultés, départements, promotions, options, années académiques. Cette gestion est hiérarchique : une faculté contient plusieurs départements, un département contient plusieurs promotions, une promotion est rattachée à une option. Cette structuration conditionne la portée des comparaisons effectuées lors de la détection de plagiat : un travail est par défaut comparé aux travaux de la même faculté, avec possibilité d'étendre la comparaison à d'autres facultés ou à l'ensemble de la plateforme.

III.3.3. Dépôt et gestion des travaux

La plateforme doit offrir une interface de dépôt des travaux académiques, avec les fonctionnalités suivantes : téléversement de fichiers au format PDF, DOCX ou TXT ; extraction automatique du texte à partir des fichiers PDF (avec OCR pour les PDF scannés) ; métadonnées associées (type de travail, sujet, étudiant, encadrant, année académique, promotion) ; contrôle de conformité (taille maximale, formats acceptés) ; archivage sécurisé du fichier original et du texte extrait. Le dépôt peut être effectué par l'étudiant lui-même, par l'enseignant encadrant, ou par un administrateur.

III.3.4. Analyse IA et détection

Le cœur fonctionnel de la plateforme est l'analyse automatique des travaux par IA. Cette analyse doit :

- Prétraiter le texte extrait (nettoyage, segmentation en phrases et en paragraphes, normalisation).
- Générer des embeddings vectoriels pour chaque segment significatif du travail analysé.
- Rechercher dans la base vectorielle les segments les plus similaires parmi les travaux antérieurs.
- Calculer un score de similarité global et des scores par segment.
- Identifier les passages potentiellement plagiés, en indiquant la source correspondante et le type de similarité (lexicale, sémantique, paraphrase).

- Produire un rapport détaillé, consultable en ligne et exportable au format PDF.

III.3.5. Rapports et tableau de bord

La plateforme doit générer automatiquement un rapport d'analyse pour chaque travail traité. Ce rapport doit comporter : un score global de similarité (pourcentage), une liste des passages identifiés comme potentiellement plagiés avec le texte source correspondant, une visualisation des segments concernés dans le document original, et des métadonnées (date d'analyse, corpus de comparaison, modèle utilisé). Un tableau de bord administratif doit offrir une vision agrégée : nombre de travaux analysés, distribution des scores de similarité, évolutions temporelles, taux de plagiat par faculté, département ou promotion.

III.3.6. Historique et traçabilité

L'ensemble des analyses effectuées doit être conservé dans un historique consultable, permettant de retracer l'évolution d'un travail, de comparer des analyses successives et d'assurer la traçabilité des décisions pédagogiques. Chaque analyse est datée, associée à l'utilisateur qui l'a déclenchée, et conserve une référence aux versions des modèles et des paramètres utilisés, afin de garantir la reproductibilité.

III.4. Analyse des besoins non fonctionnels

Au-delà des fonctionnalités attendues, la plateforme doit satisfaire un ensemble d'exigences non fonctionnelles qui conditionnent sa qualité d'usage et sa pérennité.

Tableau III.1 — Exigences non fonctionnelles de la plateforme

Exigence	Description	Cible
Performance	Temps de réponse acceptable pour les opérations utilisateur	Dépôt < 5s, consultation < 2s
Performance IA	Temps d'analyse d'un mémoire de 50 pages	< 10 minutes
Scalabilité	Capacité à traiter un volume croissant de travaux	Architecture modulaire horizontale
Disponibilité	Taux de service de la plateforme	≥ 99 % en heures ouvrables
Sécurité	Protection des données et des accès	Authentification JWT, chiffrement, RBAC

Exigence	Description	Cible
Confidentialité	Respect de la vie privée des étudiants	Hébergement local, anonymisation possible
Modularité	Capacité à étendre à d'autres facultés	Architecture en modules indépendants
Maintenabilité	Facilité de maintenance et d'évolution	Code structuré, tests, documentation
Portabilité	Indépendance vis-à-vis de l'infrastructure	Conteneurisation Docker
Accessibilité	Utilisation depuis un navigateur standard	Interface web responsive

III.5. Architecture générale de la plateforme

La plateforme est conçue selon une architecture multi-tiers classique, qui sépare clairement les responsabilités entre les couches de présentation, d'application, de services IA et de données. Cette séparation favorise la maintenabilité, l'évolutivité et la testabilité du système. Elle permet également de faire évoluer indépendamment chaque couche en fonction des besoins (par exemple, augmenter la puissance de calcul de la couche IA sans modifier l'interface utilisateur).

III.5.1. Couche présentation (frontend)

La couche présentation est constituée d'une application web développée en React. Elle est accessible depuis un navigateur moderne, sans installation préalable côté client. Elle assure l'affichage des interfaces utilisateur, la gestion des interactions, et la communication avec la couche application via des appels d'API REST. L'interface est conçue en mode responsive, pour s'adapter à différents types d'écrans (ordinateur de bureau, tablette, smartphone), ce qui est particulièrement important dans un contexte où les étudiants accèdent souvent à Internet depuis des appareils mobiles.

III.5.2. Couche application (backend)

La couche application est développée en Python avec le framework FastAPI. Elle expose une API REST qui sera consommée par le frontend. Elle gère l'authentification et l'autorisation (JWT), la validation des entrées, la coordination entre les modules métier, et

l'orchestration des appels aux services IA. Le choix de Python s'impose pour plusieurs raisons : écosystème IA mature (Sentence Transformers, FAISS, scikit-learn), performance suffisante pour un usage académique, syntaxe lisible facilitant la maintenance, large communauté de développeurs.

III.5.3. Couche services IA

La couche services IA regroupe les composants responsables du traitement intelligent des documents. Elle intègre : le module OCR (Tesseract) pour l'extraction de texte depuis les PDF scannés ; le module NLP (spaCy, NLTK) pour le prétraitement linguistique ; le module d'embeddings (Sentence Transformers) pour la vectorisation des segments ; le module de recherche vectorielle (pgvector) pour l'identification des plus proches voisins ; le module de calcul de similarité et de génération de rapports. Ces modules communiquent avec la couche application via des appels internes, et peuvent être déployés sur la même machine ou sur des serveurs dédiés en cas de besoin de scalabilité.

III.5.4. Couche données

La couche données regroupe l'ensemble des composants de persistance. Elle se compose de : une base relationnelle PostgreSQL pour les données structurées (utilisateurs, facultés, départements, travaux, analyses) ; une extension pgvector dans la même base PostgreSQL pour le stockage et la recherche des embeddings vectoriels ; un système de fichiers (ou un stockage objet) pour la conservation des fichiers originaux des travaux déposés. Le choix de regrouper le relationnel et le vectoriel dans une même base PostgreSQL simplifie l'architecture, évite l'introduction d'un système supplémentaire, et permet d'exprimer des requêtes combinant critères relationnels et similarité vectorielle en SQL.

III.6. Architecture modulaire

L'architecture de la plateforme est conçue de manière modulaire, chaque module couvrant un domaine fonctionnel cohérent et pouvant être développé, testé et déployé indépendamment. Cette modularité conditionne la capacité de la plateforme à s'étendre à d'autres facultés et universités.

Tableau III.2 — Modules fonctionnels de la plateforme

Module	Responsabilité	Principales entités
Authentification	Gestion des comptes, JWT, RBAC	User, Role, Permission
Organisation	Facultés, départements, promotions	Faculty, Department, Promotion
Utilisateurs	Étudiants, enseignants, administrateurs	Student, Teacher, Admin
Documents	Dépôt, extraction, stockage	Document, TextExtract, File
Pipeline IA	NLP, embeddings, recherche vectorielle	Embedding, Segment, Job
Analyse	Similarité, détection, scoring	Analysis, Match, Score
Rapports	Génération PDF, visualisation	Report, Section
Tableau de bord	Statistiques, agrégations	Stats, KPI
Historique	Journalisation, traçabilité	AuditLog, Event

Cette modularité présente plusieurs avantages. Elle permet l'extension progressive de la plateforme : un nouveau module peut être ajouté sans impacter les modules existants. Elle facilite la répartition du travail entre développeurs. Elle autorise des stratégies de déploiement différenciées (certains modules sur des serveurs dédiés, d'autres sur une infrastructure partagée). Enfin, elle ouvre la voie à une interconnexion future avec d'autres systèmes universitaires (systèmes de gestion scolaire, ENT, bibliothèques numériques) via des API.

III.7. Pipeline de traitement IA

Le pipeline de traitement IA constitue le cœur fonctionnel de la plateforme. Il décrit l'enchaînement des opérations réalisées depuis la soumission d'un travail académique jusqu'à la production du rapport de similarité. Chaque étape est conçue pour être autonome, testable et remplaçable, ce qui permet d'améliorer progressivement les performances du système en substituant un composant par une version plus efficace.

III.7.1. Soumission du document

Le pipeline démarre lorsqu'un utilisateur dépose un travail académique via l'interface web. Le fichier (PDF, DOCX ou TXT) est téléversé vers le serveur, où il est stocké de manière sécurisée. Les métadonnées associées (type de travail, étudiant, encadrant, promotion, année académique) sont enregistrées dans la base relationnelle. Un identifiant unique est attribué au travail, qui sera utilisé tout au long du pipeline.

III.7.2. Extraction du texte

L'étape suivante consiste à extraire le texte du document déposé. Pour les fichiers DOCX et TXT, l'extraction est directe. Pour les fichiers PDF, plusieurs cas sont possibles : si le PDF contient une couche de texte sélectionnable, l'extraction est réalisée par une bibliothèque comme PyPDF2 ou pdfplumber ; si le PDF est scanné (image), une étape d'OCR avec Tesseract est nécessaire. Le texte extrait est nettoyé (suppression des en-têtes et pieds de page, normalisation des caractères) et stocké dans la base, associé au travail.

III.7.3. Prétraitement NLP

Le texte extrait est ensuite soumis à un prétraitement NLP. Cette étape comprend : la segmentation en phrases (sentence splitting), la tokenisation, la normalisation (mise en minuscules, suppression de la ponctuation excédentaire), la suppression des stop words pour certaines analyses, et éventuellement la lemmatisation. Le résultat est une liste de segments (phrases ou groupes de phrases) prêts à être vectorisés. La taille des segments est un paramètre important : des segments trop courts produisent des embeddings peu informatifs, des segments trop longs diluent l'information sémantique. Une taille de 1 à 3 phrases est généralement retenue.

III.7.4. Génération des embeddings

Chaque segment est ensuite converti en un vecteur d'embedding par le modèle Sentence-BERT. Le modèle utilisé (distiluse-base-multilingual-cased-v1) produit des vecteurs de dimension 512, capables de capturer le sens des phrases en français et en anglais. Les embeddings générés sont stockés dans la base vectorielle (pgvector), associés à l'identifiant du travail et à la position du segment dans le document original.

III.7.5. Recherche vectorielle

Pour chaque segment du travail analysé, la plateforme recherche dans la base vectorielle les segments les plus similaires parmi l'ensemble des travaux antérieurs. Cette recherche est effectuée à l'aide de l'index HNSW de pgvector, qui permet une recherche approximative des k plus proches voisins en quelques millisecondes, même sur des bases de plusieurs millions de vecteurs. La recherche peut être filtrée par faculté, département ou promotion, selon la configuration choisie.

III.7.6. Calcul de similarité

Pour chaque paire de segments identifiés comme proches, la similarité cosinus est calculée précisément. Les paires dont la similarité dépasse un seuil prédéfini (par défaut 0,80) sont retenues comme des correspondances potentielles. Un score global de similarité est calculé pour le travail, en agrégeant les correspondances : ce score représente la proportion du travail qui présente des similarités significatives avec des sources existantes.

III.7.7. Détection et classification

Les correspondances identifiées sont ensuite classées par type : copier-coller strict (similarité lexicale très élevée), paraphrase (similarité sémantique élevée avec similarité lexicale plus faible), reformulation (similarité sémantique élevée avec structure syntaxique différente). Cette classification aide l'enseignant à évaluer la gravité des similarités détectées. Les faux positifs potentiels (citations correctement référencées, passages standards comme les définitions) sont signalés pour révision humaine.

III.7.8. Génération du rapport

Enfin, un rapport détaillé est généré automatiquement. Il comporte : un résumé exécutif (score global, nombre de correspondances, classification), une liste détaillée des passages identifiés avec leur source et le type de similarité, une visualisation du document original avec surlignage des passages concernés, et des recommandations pour l'évaluation. Le rapport est consultable en ligne dans l'interface et exportable au format PDF.

III.8. Base documentaire relationnelle

La base documentaire relationnelle est structurée autour des entités principales suivantes : Faculty (faculté), Department (département), Promotion, User (utilisateur, avec sous-types Student, Teacher, Admin), Document (travail académique), Analysis (analyse), Match (correspondance détectée). Les relations entre ces entités modélisent l'organisation académique et le cycle de vie des travaux.

Le schéma relationnel simplifié est le suivant : une Faculty contient plusieurs Departments, un Department contient plusieurs Promotions, une Promotion contient plusieurs Students. Un Document est rattaché à un Student, à un Teacher (encadrant), à une Promotion et à une Faculty. Une Analysis est associée à un Document et à un User (celui qui a déclenché l'analyse). Plusieurs Matches sont associés à une Analysis, chaque Match liant un segment du travail analysé à un segment d'un travail antérieur.

Des contraintes d'intégrité garantissent la cohérence des données : clés étrangères entre les tables, unicité des identifiants, contraintes de domaine (par exemple, le score de similarité doit être compris entre 0 et 1). Des index sont créés sur les champs les plus sollicités pour optimiser les performances (par exemple, index sur l'identifiant de l'étudiant, sur la date d'analyse, sur le score de similarité).

III.9. Base vectorielle

La base vectorielle est intégrée à la base relationnelle via l'extension pgvector. Une table spécifique, DocumentEmbeddings, stocke les embeddings de tous les segments de tous les travaux. Cette table contient les champs suivants : identifiant unique de l'embedding, identifiant du document source, position du segment dans le document, texte du segment, vecteur d'embedding (type vector(512)), date de création.

Un index HNSW (Hierarchical Navigable Small World) est créé sur le champ vectoriel pour permettre une recherche rapide des plus proches voisins. Les paramètres de l'index (m, ef_construction, ef_search) sont choisis pour offrir un bon compromis entre vitesse de recherche et précision. Des requêtes SQL spécifiques permettent de rechercher les segments les plus similaires à un vecteur donné, en filtrant éventuellement par faculté ou département.

L'avantage d'intégrer la base vectorielle à PostgreSQL est triple : pas de système supplémentaire à maintenir, cohérence transactionnelle entre les données relationnelles et vectorielles, possibilité d'exprimer des requêtes combinant les deux types de données.

L'inconvénient est une performance potentiellement inférieure à des systèmes vectoriels dédiés (FAISS, Milvus) pour des volumes très importants, mais ce compromis est acceptable dans le contexte d'une plateforme universitaire nationale où le volume de documents reste maîtrisé.

III.10. Conception de l'API REST

L'API REST constitue l'interface entre le frontend et le backend. Elle expose un ensemble d'endpoints organisés par ressource, suivant les principes REST. Les principaux endpoints sont les suivants :

Tableau III.3 — Principaux endpoints de l'API REST

Endpoint	Méthode	Description	Authentification
/api/auth/login	POST	Authentification et génération JWT	Publique
/api/students	GET/POST	Liste et création d'étudiants	Admin
/api/teachers	GET/POST	Liste et création d'enseignants	Admin
/api/faculties	GET/POST	Liste et création de facultés	Super-admin
/api/documents	GET/POST	Liste et dépôt de travaux	Étudiant/Enseignant
/api/documents/{id}/analyze	POST	Déclenchement d'une analyse	Enseignant/Admin
/api/analyses/{id}	GET	Récupération d'un rapport	Enseignant/Admin
/api/analyses/{id}/pdf	GET	Export PDF d'un rapport	Enseignant/Admin
/api/dashboard/stats	GET	Statistiques agrégées	Admin

L'authentification est gérée par JWT (JSON Web Tokens). À la connexion, l'utilisateur reçoit un token signé, qu'il doit inclure dans l'en-tête Authorization des requêtes suivantes. Les autorisations sont gérées par un mécanisme RBAC (Role-Based Access Control), qui associe à chaque rôle un ensemble de permissions. Les réponses de l'API sont au format JSON, et les erreurs suivent les codes HTTP standards (200 pour succès, 400 pour erreur

client, 401 pour non authentifié, 403 pour non autorisé, 404 pour introuvable, 500 pour erreur serveur).

III.11. Modélisation UML

III.11.1. Diagramme de cas d'utilisation

Le diagramme de cas d'utilisation identifie les principaux acteurs du système (Étudiant, Enseignant, Administrateur, Système IA) et les cas d'utilisation associés. L'Étudiant peut : s'authentifier, déposer un travail, consulter ses rapports. L'Enseignant peut : s'authentifier, consulter les travaux de ses étudiants, déclencher une analyse, consulter les rapports, valider ou rejeter un travail. L'Administrateur peut : gérer les utilisateurs, gérer l'organisation académique, consulter le tableau de bord. Le Système IA intervient automatiquement lors du dépôt (extraction du texte) et lors du déclenchement d'une analyse (pipeline complet).

III.11.2. Diagramme de classes

Le diagramme de classes modélise les entités principales du système et leurs relations. Les classes identifiées sont : User (avec ses sous-classes Student, Teacher, Admin), Faculty, Department, Promotion, Document, TextExtract, Segment, Embedding, Analysis, Match, Report. Les associations entre ces classes traduisent les relations identifiées dans le schéma relationnel : une Faculty contient plusieurs Departments, un Document a plusieurs Segments, une Analysis a plusieurs Matches, etc. Les attributs et méthodes de chaque classe sont définis selon les principes de la programmation orientée objet.

III.11.3. Diagramme de séquences

Le diagramme de séquences décrit l'enchaînement des interactions lors du dépôt et de l'analyse d'un travail. La séquence principale est la suivante : l'Étudiant dépose un travail via le Frontend ; le Frontend envoie une requête POST /api/documents au Backend ; le Backend stocke le fichier, crée l'entrée en base, et déclenche l'extraction du texte via le module OCR/NLP ; le texte extrait est stocké ; un message de succès est renvoyé à l'utilisateur. Lorsqu'une analyse est déclenchée par l'Enseignant, le Backend orchestre le pipeline : prétraitement NLP, génération des embeddings, recherche vectorielle, calcul de similarité, génération du rapport. Le rapport est stocké en base et notifié à l'enseignant.

III.12. Diagrammes d'architecture logicielle

Au-delà des diagrammes UML classiques, l'architecture de la plateforme est décrite par des diagrammes d'architecture logicielle qui mettent en évidence la répartition des composants et les flux de données. Le diagramme d'architecture globale présente les quatre couches (présentation, application, services IA, données) et leurs interactions. Le diagramme de déploiement décrit la répartition des composants sur les serveurs (serveur web, serveur d'application, serveur de base de données, éventuellement serveur IA dédié). Le diagramme de flux de données illustre le parcours d'un travail depuis le dépôt jusqu'au rapport final.

Ces diagrammes ne sont pas de simples illustrations : ils constituent la référence technique du projet, à partir de laquelle les développements sont organisés et les évolutions sont planifiées. Ils sont versionnés et mis à jour à chaque évolution significative de l'architecture.

III.13. Conclusion

Ce chapitre a présenté la conception détaillée de la plateforme web intelligente. L'architecture multi-tiers, la modularité fonctionnelle, le pipeline IA en huit étapes, la base relationnelle PostgreSQL étendue par pgvector, et l'API REST constituent les piliers techniques de la solution. La modélisation UML et les diagrammes d'architecture logicielle fournissent une représentation rigoureuse du système, à la fois pour la communication avec les parties prenantes et pour l'orientation des développements.

Le chapitre suivant sera consacré à l'implémentation concrète de cette conception, à l'expérimentation de la plateforme sur un corpus pilote de travaux académiques de la Faculté des Sciences, et à l'évaluation quantitative de ses performances. Il présentera la stack technique effectivement mise en œuvre, les extraits de code clés du pipeline, les captures d'écran de l'interface, et l'analyse des résultats d'évaluation.

CHAPITRE IV — IMPLÉMENTATION, EXPÉRIMENTATION ET ÉVALUATION

IV.1. Introduction

Ce chapitre présente l'implémentation concrète de la plateforme conçue au chapitre précédent, son expérimentation sur un corpus pilote de travaux académiques de la Faculté des Sciences, et l'évaluation quantitative de ses performances. Il détaille la stack technique mise en œuvre, les principaux extraits de code du pipeline IA, les captures d'écran de l'interface, et l'analyse des résultats obtenus sur le corpus pilote. L'objectif est de valider l'hypothèse formulée en introduction, à savoir que l'approche sémantique fondée sur les embeddings vectoriels permet une détection plus efficace que les approches lexicales classiques.

L'évaluation s'appuie sur un protocole rigoureux : constitution d'un corpus pilote annoté, définition d'une baseline lexicale, exécution des deux approches sur le même corpus, calcul des métriques de performance (précision, rappel, F1-score, courbe ROC), analyse comparative. Les limites observées et les pistes d'amélioration sont discutées en fin de chapitre.

IV.2. Stack technique

La stack technique retenue résulte d'un compromis entre performance, maturité, coût, disponibilité des compétences et adéquation au contexte. Elle repose majoritairement sur des technologies open source, ce qui élimine les contraintes de licence et garantit la maîtrise du code.

Tableau IV.1 — Stack technique de la plateforme

Composant	Technologie	Version	Rôle
Langage backend	Python	3.11	Logique métier, orchestration
Framework API	FastAPI	0.110+	API REST asynchrone
ORM	SQLAlchemy	2.0	Accès base de données
Base relationnelle	PostgreSQL	16	Stockage données structurées

Composant	Technologie	Version	Rôle
Extension vectorielle	pgvector	0.7+	Recherche vectorielle
Modèle embeddings	Sentence Transformers	2.7+	Vectorisation sémantique
Modèle spécifique	distiluse-base-multilingual-cased-v1	—	Embeddings multilingues FR/EN
OCR	Tesseract + pytesseract	5.3+	Extraction texte PDF scannés
Extraction PDF	pdfplumber	0.10+	Extraction texte PDF natifs
Prétraitement NLP	spaCy (fr_core_news_sm)	3.7+	Tokenisation, lemmatisation
Frontend	React + Vite + TypeScript	React 18	Interface web
UI Frontend	Tailwind CSS + shadcn/ui	—	Composants interface
Authentification	JWT (PyJWT)	—	Tokens d'authentification
Génération PDF	ReportLab	4.0+	Rapports PDF
Conteneurisation	Docker + Docker Compose	—	Déploiement reproductible
Serveur web	Nginx	—	Reverse proxy, fichiers statiques

Cette stack présente plusieurs avantages. Côté backend, Python et FastAPI offrent un excellent compromis entre productivité de développement, performance et écosystème IA. Côté frontend, React et TypeScript garantissent une interface moderne et maintenable. Côté données, PostgreSQL avec pgvector évite l'introduction d'un système vectoriel dédié, simplifiant le déploiement et la maintenance. La conteneurisation Docker assure la reproductibilité du déploiement sur différentes infrastructures.

IV.3. Frontend

Le frontend est une application React monopage (SPA) développée en TypeScript avec Vite comme outil de build. L'interface utilise Tailwind CSS pour le style et shadcn/ui pour les composants réutilisables (boutons, formulaires, tableaux, dialogues). Le routing est

assuré par React Router, et la gestion d'état par Zustand. Les appels API sont effectués via axios, avec un interceptor qui ajoute automatiquement le token JWT aux requêtes authentifiées.

Les principales vues de l'interface sont : la page de connexion, le tableau de bord (selon le rôle de l'utilisateur), la liste des travaux, le formulaire de dépôt, la visualisation d'un rapport d'analyse (avec surlignage des passages similaires), et les écrans d'administration (gestion des utilisateurs, des facultés, des départements). L'interface est conçue en mode responsive, pour s'adapter aux ordinateurs de bureau comme aux tablettes et smartphones, ce qui est crucial dans un contexte où l'accès mobile à Internet est dominant.

La visualisation des rapports d'analyse constitue un élément clé de l'expérience utilisateur. Elle permet à l'enseignant de naviguer dans le document analysé, de voir les passages surlignés selon le type de similarité détectée (vert pour les paraphrases, orange pour les reformulations, rouge pour le copier-coller strict), et d'accéder en un clic au texte source correspondant. Une vue comparative côte à côte facilite l'évaluation manuelle par l'enseignant, qui conserve la responsabilité finale du jugement sur l'existence ou non d'un plagiat.

IV.4. Backend

Le backend est développé en Python 3.11 avec FastAPI. Il suit une architecture en couches : routers (gestion des requêtes HTTP), services (logique métier), repositories (accès aux données), models (entités SQLAlchemy). L'authentification est gérée par un mécanisme JWT, avec un middleware qui vérifie le token et l'autorisation à chaque requête. La validation des entrées est assurée par Pydantic, qui génère automatiquement la documentation OpenAPI de l'API.

L'orchestration du pipeline IA est implémentée sous forme d'une tâche asynchrone, déclenchée par un endpoint POST /api/documents/{id}/analyze. Cette tâche est exécutée par un worker Celery (ou un simple asyncio en fonction de la charge), ce qui permet à l'utilisateur de ne pas attendre la fin de l'analyse pour recevoir une réponse HTTP. L'utilisateur est notifié de la disponibilité du rapport via une notification in-app ou par email. Cette architecture asynchrone est essentielle compte tenu du temps de traitement du pipeline (plusieurs minutes pour un mémoire de 50 pages).

IV.5. Base relationnelle

La base relationnelle PostgreSQL contient l'ensemble des données structurées de la plateforme. Le schéma comprend une vingtaine de tables, dont les principales sont : users, faculties, departments, promotions, students, teachers, documents, text_extracts, segments, analyses, matches, reports, audit_logs. Les contraintes d'intégrité référentielle garantissent la cohérence des données, et des index sont créés sur les champs les plus sollicités pour optimiser les performances.

Le script de création de la base définit l'extension pgvector et la table des embeddings :

```
-- Activation de l'extension pgvector
CREATE EXTENSION IF NOT EXISTS vector;

-- Table des embeddings
CREATE TABLE document_embeddings (
  id BIGSERIAL PRIMARY KEY,
  document_id BIGINT NOT NULL REFERENCES documents(id) ON DELETE CASCADE,
  segment_index INT NOT NULL,
  segment_text TEXT NOT NULL,
  embedding vector(512) NOT NULL,
  created_at TIMESTAMPTZ DEFAULT NOW()
);

-- Index HNSW pour la recherche vectorielle rapide
CREATE INDEX idx_embeddings_hnsw
ON document_embeddings
USING hnsw (embedding vector_cosine_ops)
WITH (m = 16, ef_construction = 64);

-- Requête type : k plus proches voisins d'un vecteur donné
SELECT document_id, segment_index, segment_text,
  1 - (embedding <=> $1) AS similarity
FROM document_embeddings
WHERE document_id != $2 -- exclure le document lui-même
ORDER BY embedding <=> $1
LIMIT 10;
```

IV.6. Base vectorielle

La base vectorielle est intégrée à PostgreSQL via pgvector, comme décrit à la section précédente. Le choix de l'index HNSW (Hierarchical Navigable Small World) se justifie par ses excellentes performances sur des bases de plusieurs millions de vecteurs, avec un compromis favorable entre vitesse de recherche et précision. Les paramètres retenus (m=16, ef_construction=64) correspondent aux valeurs recommandées par la documentation pgvector pour un usage général.

Le type vector(512) correspond à la dimension des embeddings produits par le modèle distiluse-base-multilingual-cased-v1. Cette dimension relativement réduite (comparée aux 768 ou 1024 de modèles plus lourds) offre un bon compromis entre qualité sémantique et coût de stockage/recherche. Pour un corpus de 100 000 travaux académiques de 50 pages en moyenne, avec environ 200 segments par travail, la base vectorielle contiendrait 20 millions de vecteurs, soit environ 40 Go de stockage — un volume tout à fait gérable sur un serveur moderne.

IV.7. OCR et extraction du texte

L'extraction du texte depuis les documents déposés est une étape critique du pipeline, car la qualité du texte extrait conditionne directement la qualité des analyses ultérieures. Deux cas sont distingués : les PDF natifs (avec couche de texte sélectionnable) et les PDF scannés (images).

Pour les PDF natifs, la bibliothèque pdfplumber est utilisée. Elle permet d'extraire le texte avec une bonne qualité, en préservant la structure (paragraphe, colonnes). Pour les PDF scannés, une étape d'OCR est nécessaire, réalisée avec Tesseract via le wrapper Python pytesseract. L'OCR est précédé d'une étape de prétraitement de l'image (conversion en niveaux de gris, binarisation) pour améliorer la qualité de la reconnaissance.

L'extrait de code suivant illustre la logique d'extraction :

```
import pdfplumber
import pytesseract
from pdf2image import convert_from_path
from PIL import Image
import io

def extract_text_from_pdf(pdf_path: str) -> str:
    """Extrait le texte d'un PDF, avec fallback OCR si nécessaire."""
    text_parts = []
    with pdfplumber.open(pdf_path) as pdf:
        for page_num, page in enumerate(pdf.pages):
            text = page.extract_text() or ""
            if len(text.strip()) < 50:
                # Page probablement scannée -> OCR
                images = convert_from_path(pdf_path,
                                           first_page=page_num + 1,
                                           last_page=page_num + 1,
                                           dpi=300)
                for img in images:
                    text = pytesseract.image_to_string(
                        img, lang="fra+eng"
                    )
            text_parts.append(text)
```

```
return "\n\n".join(text_parts)
```

IV.8. Pipeline NLP

Le prétraitement NLP est implémenté avec spaCy (modèle fr_core_news_sm pour le français) et NLTK en complément. Il comprend les étapes suivantes : nettoyage du texte (suppression des caractères parasites, normalisation des espaces), segmentation en phrases, tokenisation, lemmatisation optionnelle, suppression des segments trop courts (moins de 5 mots) ou non informatifs (références bibliographiques, numéros de page).

L'implémentation du pipeline NLP est illustrée ci-dessous :

```
import spacy
import re
from typing import List

nlp = spacy.load("fr_core_news_sm")

def clean_text(text: str) -> str:
    """Nettoie le texte extrait."""
    text = re.sub(r"\n{3,}", "\n\n", text)
    text = re.sub(r"[\t]+", " ", text)
    text = re.sub(r"[\d+]", "", text) # refs
    return text.strip()

def segment_sentences(text: str,
                      min_words: int = 5) -> List[str]:
    """Segmentation en phrases avec filtrage."""
    doc = nlp(text)
    segments = []
    for sent in doc.sents:
        tokens = [t for t in sent if not t.is_space]
        if len(tokens) >= min_words:
            segments.append(sent.text.strip())
    return segments

def preprocess_document(text: str) -> List[str]:
    """Pipeline complet de prétraitement."""
    cleaned = clean_text(text)
    segments = segment_sentences(cleaned)
    return segments
```

IV.9. Génération des embeddings

La génération des embeddings est réalisée avec Sentence Transformers, en utilisant le modèle distiluse-base-multilingual-cased-v1. Ce modèle multilingue couvre 50+ langues dont le français et l'anglais, ce qui est essentiel pour traiter des travaux académiques qui

peuvent citer des sources dans les deux langues. Sa taille réduite (environ 500 Mo) permet un déploiement sur des serveurs aux ressources limitées, et sa vitesse d'inférence est compatible avec un traitement en ligne.

L'extrait de code suivant montre la génération des embeddings :

```
from sentence_transformers import SentenceTransformer
import numpy as np

# Chargement du modèle (mis en cache après le premier appel)
model = SentenceTransformer(
    "distiluse-base-multilingual-cased-v1"
)

def generate_embeddings(segments: list[str]) -> np.ndarray:
    """Génère les embeddings d'une liste de segments."""
    embeddings = model.encode(
        segments,
        batch_size=32,
        show_progress_bar=False,
        convert_to_numpy=True,
        normalize_embeddings=True, # L2-norm pour cosinus
    )
    return embeddings

# Exemple d'usage
segments = ["Le plagiat est une faute académique.",
            "L'intelligence artificielle transforme la détection."]
emb = generate_embeddings(segments)
print(emb.shape) # (2, 512)
```

IV.10. Recherche vectorielle

La recherche vectorielle est effectuée directement en SQL via pgvector. Pour chaque segment du travail analysé, on recherche les k segments les plus similaires parmi les travaux antérieurs, en excluant le travail lui-même et en filtrant éventuellement par faculté. L'opérateur de distance cosinus de pgvector calcule l'écart angulaire entre vecteurs ; la similarité s'obtient par 1 moins la distance.

La requête SQL type, exécutée pour chaque segment du travail analysé, sélectionne les dix segments les plus proches dans la base vectorielle, joint la table des documents pour récupérer les métadonnées (titre, auteur), filtre sur la faculté et exclut le document en cours d'analyse. Les résultats sont triés par distance croissante et limités aux dix meilleurs. Cette requête est exécutée en boucle sur l'ensemble des segments du travail, puis les résultats sont agrégés au niveau du document pour produire le score global.

L'implémentation Python de la recherche vectorielle, utilisée côté backend, est présentée ci-dessous :

```
# Recherche vectorielle via SQLAlchemy + pgvector
from sqlalchemy import text

def search_similar_segments(
    session,
    query_embedding: list[float],
    current_doc_id: int,
    faculty_id: int,
    k: int = 10,
    threshold: float = 0.80,
) -> list[dict]:
    """Recherche les k segments les plus similaires au vecteur requête."""
    sql = text("""
        SELECT
            e.document_id,
            e.segment_index,
            e.segment_text,
            d.title AS source_title,
            d.author_name AS source_author,
            1 - (e.embedding <=> :qv) AS similarity
        FROM document_embeddings e
        JOIN documents d ON d.id = e.document_id
        WHERE e.document_id != :cur
            AND d.faculty_id = :fac
        ORDER BY e.embedding <=> :qv
        LIMIT :k
    """)
    rows = session.execute(sql, {
        "qv": str(query_embedding),
        "cur": current_doc_id,
        "fac": faculty_id,
        "k": k,
    }).fetchall()
    return [dict(r._mapping) for r in rows
            if r.similarity >= threshold]
```

IV.11. Calcul de similarité et scoring

Le calcul de similarité agrège les résultats de la recherche vectorielle pour produire un score global par travail. Le score global est défini comme la proportion de segments du travail analysé qui présentent au moins une correspondance significative (similarité $\geq$ seuil) avec un travail antérieur. Un score par paire de documents est également calculé, indiquant le pourcentage du travail analysé qui présente des similarités avec chaque travail source identifié.

Le code Python suivant illustre la logique de scoring :

```
from collections import defaultdict
```

```

def compute_similarity_score(
    segments: list[str],
    matches: list[dict],
    threshold: float = 0.80
) -> dict:
    """Calcule le score de similarité global et par source."""
    matched_segments = set()
    by_source = defaultdict(lambda: {"count": 0, "max_sim": 0.0})

    for m in matches:
        if m["similarity"] >= threshold:
            matched_segments.add(m["query_segment_idx"])
            src = m["source_document_id"]
            by_source[src]["count"] += 1
            by_source[src]["max_sim"] = max(
                by_source[src]["max_sim"],
                m["similarity"]
            )

    global_score = len(matched_segments) / len(segments)
    return {
        "global_score": global_score,
        "matched_segments": len(matched_segments),
        "total_segments": len(segments),
        "by_source": dict(by_source),
    }

```

IV.12. Détection des paraphrases

La détection des paraphrases constitue l'avantage différenciant de l'approche sémantique par rapport aux approches lexicales. Une paraphrase est identifiée lorsque la similarité sémantique (cosinus des embeddings) est élevée, mais que la similarité lexicale (mesurée par exemple par le coefficient de Jaccard sur les ensembles de mots) est plus faible. Cette dissociation permet de distinguer les copier-coller stricts (similarité lexicale ET sémantique élevées) des paraphrases (similarité sémantique élevée, similarité lexicale modérée) et des reformulations plus libres (similarité sémantique élevée, similarité lexicale faible).

La classification des correspondances est implémentée comme suit :

```

def classify_match(query_text: str,
                  source_text: str,
                  semantic_sim: float) -> str:
    """Classifie une correspondance détectée."""
    q_tokens = set(query_text.lower().split())
    s_tokens = set(source_text.lower().split())
    jaccard = len(q_tokens & s_tokens) / len(q_tokens | s_tokens)

```

```

if semantic_sim >= 0.95 and jaccard >= 0.85:
    return "copy_paste"
elif semantic_sim >= 0.85 and jaccard >= 0.50:
    return "paraphrase"
elif semantic_sim >= 0.80:
    return "reformulation"
else:
    return "weak_match"

```

IV.13. Génération automatique des rapports PDF

Les rapports d'analyse sont générés automatiquement au format PDF à l'aide de la bibliothèque ReportLab. Le rapport comprend : une page de garde avec les métadonnées du travail analysé (titre, auteur, encadrant, faculté, date d'analyse), un résumé exécutif (score global, nombre de correspondances, classification), une liste détaillée des passages identifiés comme potentiellement plagiés (texte du passage, texte source, type de similarité, score), et une visualisation du document original avec surlignage des passages concernés.

Le rapport est conçu pour être exploitable directement par l'enseignant ou le jury. Il fournit l'information nécessaire à une décision éclairée, sans remplacer le jugement humain. L'enseignant conserve la responsabilité de qualifier les similarités détectées : un passage identifié comme potentiellement plagié peut en réalité être une citation correctement référencée, une définition standard, ou une reprise légitime d'un cadre conceptuel. Le rapport signale ces cas pour faciliter la révision humaine.

IV.14. Tests

Plusieurs niveaux de tests ont été mis en place pour garantir la qualité et la fiabilité de la plateforme. Les tests unitaires (avec pytest) vérifient le bon fonctionnement de chaque fonction isolément : fonctions de prétraitement NLP, génération d'embeddings, calcul de similarité, classification des correspondances. Les tests d'intégration valident la coordination entre modules : extraction de texte depuis un PDF, génération d'embeddings, recherche vectorielle en base. Les tests fonctionnels (end-to-end) simulent des scénarios complets : dépôt d'un travail, déclenchement d'une analyse, consultation du rapport.

Une attention particulière a été portée à la constitution d'un jeu de tests représentatif, incluant : des travaux authentiques (sans plagiat), des travaux avec copier-coller strict depuis un mémoire antérieur, des travaux avec paraphrases, des travaux avec traduction depuis une source anglaise, et des travaux avec des passages générés par un LLM. Ce jeu de tests permet

de valider la capacité de la plateforme à détecter les différents types de plagiat, et à éviter les faux positifs sur les travaux authentiques.

IV.15. Évaluation sur corpus pilote

IV.15.1. Constitution du corpus pilote

Le corpus pilote a été constitué à partir de mémoires et de TFC effectivement déposés à la Faculté des Sciences de l'Université de Kinshasa au cours des cinq dernières années académiques. Il comprend 150 travaux académiques, répartis comme suit : 100 travaux authentiques (constituant la base de référence pour les comparaisons), 30 travaux contenant des plagiatats artificiellement insérés (copier-coller, paraphrases, traductions) pour lesquels les cas de plagiat sont annotés manuellement, et 20 travaux avec des passages générés par un LLM (ChatGPT) pour évaluer la capacité de détection du plagiat par IA.

IV.15.2. Baseline lexicale

Une baseline lexicale a été implémentée pour servir de référence à l'évaluation. Elle repose sur la comparaison de n-grammes de mots (3-grammes par défaut) entre les travaux, avec calcul d'un score de similarité de Jaccard. Cette baseline représente l'état de l'art des approches classiques de détection lexicale, et permet de mesurer l'apport de l'approche sémantique.

IV.15.3. Métriques d'évaluation

Les performances sont évaluées avec les métriques standard de classification binaire au niveau du segment : précision (proportion des segments identifiés comme plagiés qui le sont effectivement), rappel (proportion des segments effectivement plagiés qui sont identifiés comme tels), F1-score (moyenne harmonique des deux). La courbe ROC (Receiver Operating Characteristic) et l'aire sous la courbe (AUC) sont également calculées pour évaluer la capacité discriminante du système à différents seuils.

IV.15.4. Résultats quantitatifs

Tableau IV.2 — Performances comparées des approches sur le corpus pilote (segments annotés)

Approche	Précision	Rappel	F1-score	AUC
Baseline lexicale (3-grammes, Jaccard)	0,91	0,52	0,66	0,72
Approche sémantique (Sentence-BERT, cosinus)	0,84	0,83	0,84	0,89
Approche hybride (lexicale + sémantique)	0,88	0,87	0,87	0,92

Les résultats obtenus valident l'hypothèse formulée en introduction. L'approche sémantique fondée sur les embeddings Sentence-BERT obtient un F1-score de 0,84, contre 0,66 pour la baseline lexicale, soit une amélioration de 27 % en valeur relative. Le gain le plus significatif concerne le rappel (0,83 contre 0,52), ce qui confirme que l'approche sémantique détecte des cas de plagiat (notamment des paraphrases et des reformulations) qui échappent à la baseline lexicale. La précision est légèrement inférieure (0,84 contre 0,91), ce qui s'explique par un nombre plus élevé de faux positifs : l'approche sémantique identifie des similarités de sens qui peuvent ne pas constituer un plagiat avéré (par exemple, des passages reprenant des concepts standards).

L'approche hybride, qui combine les deux méthodes en prenant l'union des correspondances détectées, obtient les meilleures performances (F1 = 0,87, AUC = 0,92). Elle sera retenue comme configuration par défaut de la plateforme, l'approche purement sémantique pouvant être activée pour des analyses approfondies ciblées.

IV.15.5. Analyse par type de plagiat

Tableau IV.3 — Rappel par type de plagiat

Type de plagiat	Baseline lexicale (rappel)	Approche sémantique (rappel)
Copier-coller strict	0,95	0,93
Paraphrase manuelle	0,28	0,79

Type de plagiat	Baseline lexicale (rappel)	Approche sémantique (rappel)
Paraphrase par LLM	0,12	0,71
Traduction non sourcée	0,05	0,62
Plagiat par IA générative	0,00	0,18

Cette analyse détaillée confirme l'apport différenciant de l'approche sémantique. Sur le copier-coller strict, les deux approches sont équivalentes (la baseline lexicale est même légèrement supérieure, ce qui est attendu). Sur les paraphrases manuelles, l'approche sémantique détecte près de 80 % des cas, contre moins de 30 % pour la baseline. Sur les paraphrases produites par LLM, l'écart est encore plus marqué : 71 % contre 12 %. Les traductions non sourcées, qui échappent quasi totalement à la baseline lexicale, sont détectées dans 62 % des cas par l'approche sémantique, grâce au caractère multilingue du modèle d'embeddings. Enfin, le plagiat par IA générative reste un défi : aucun des deux approches ne le détecte de manière satisfaisante, mais l'approche sémantique obtient un rappel non nul (18 %), ce qui suggère que les embeddings peuvent capturer certaines régularités des textes générés par LLM.

IV.16. Performances et scalabilité

Les performances de la plateforme ont été mesurées sur un serveur de test (4 cœurs CPU, 16 Go RAM, SSD 200 Go, GPU non requis pour l'inférence Sentence-BERT multilingue). Le temps de traitement moyen d'un mémoire de 50 pages (environ 200 segments) est de 7 minutes 30 secondes, dont 4 minutes pour l'OCR et l'extraction du texte, 1 minute pour le prétraitement NLP, 1 minute pour la génération des embeddings, et 1 minute 30 secondes pour la recherche vectorielle et le calcul de similarité. Ce temps de traitement est compatible avec un usage asynchrone, où l'utilisateur est notifié de la disponibilité du rapport.

La scalabilité de la plateforme a été évaluée en termes de volume de travaux stockés et de temps de recherche. Avec 10 000 travaux en base (environ 2 millions de segments), le temps de recherche vectorielle pour un travail complet (200 requêtes) reste inférieur à 2 minutes, grâce à l'index HNSW. Au-delà de 100 000 travaux, une architecture distribuée (avec un serveur de base de données dédié et éventuellement un shard par faculté) sera nécessaire pour maintenir des performances acceptables.

IV.17. Limites observées

Plusieurs limites ont été identifiées lors de l'expérimentation, qui ouvrent des perspectives d'amélioration. Premièrement, la qualité de l'OCR sur les PDF scannés de mauvaise qualité reste un facteur limitant : un texte mal extrait conduit à des embeddings dégradés et à des faux négatifs. Des solutions d'OCR plus avancées (comme EasyOCR ou des modèles basés sur Transformer) pourront être intégrées en complément de Tesseract.

Deuxièmement, la détection du plagiat par IA générative reste un défi non résolu, avec un rappel de seulement 18 %. Les approches émergentes (classificateurs supervisés spécialisés, métriques de perplexité, analyse stylométrique) devront être intégrées pour améliorer ce point. La rapidité d'évolution des LLM rend toutefois cette lutte particulièrement difficile, et il convient d'être lucide sur les limites actuelles de toute solution technique à ce problème.

Troisièmement, le corpus pilote de 150 travaux, bien que suffisant pour valider la preuve de concept, reste limité pour généraliser les résultats à grande échelle. Une expérimentation à plus large échelle, sur plusieurs années académiques et plusieurs facultés, sera nécessaire pour confirmer les performances et affiner les seuils de décision.

Quatrièmement, les embeddings multilingues, bien qu'efficaces, ne sont pas exempts de biais linguistiques : les similarités entre textes en français peuvent être légèrement surestimées par rapport aux similarités entre textes français et anglais, en raison de la distribution des données d'entraînement du modèle. Un fine-tuning sur un corpus académique francophone pourrait améliorer la qualité des embeddings pour le contexte cible.

IV.18. Conclusion

Ce chapitre a présenté l'implémentation concrète de la plateforme web intelligente, son expérimentation sur un corpus pilote de 150 travaux académiques de la Faculté des Sciences, et l'évaluation quantitative de ses performances. Les résultats obtenus valident l'hypothèse formulée en introduction : l'approche sémantique fondée sur les embeddings Sentence-BERT améliore significativement la détection du plagiat par rapport à la baseline lexicale, avec un gain de 27 % sur le F1-score et un gain particulièrement marqué sur le rappel des paraphrases et des traductions.

L'approche hybride, combinant méthode lexicale et méthode sémantique, obtient les meilleures performances ($F1 = 0,87$, $AUC = 0,92$) et sera retenue comme configuration par

défaut. Les limites identifiées, notamment sur la détection du plagiat par IA générative et sur la qualité de l'OCR pour les documents dégradés, ouvrent des perspectives claires d'amélioration, qui seront discutées dans la conclusion générale.

CONCLUSION GÉNÉRALE

Au terme de ce travail consacré au développement d'une plateforme web basée sur l'Intelligence Artificielle pour la détection automatique du plagiat dans les travaux académiques, il convient de revenir sur les principaux résultats obtenus, les limites observées et les perspectives qu'ouvre cette recherche. L'ambition initiale était de proposer une alternative moderne aux outils classiques de détection du plagiat, en s'appuyant sur les techniques récentes d'Intelligence Artificielle et en tenant compte des spécificités du contexte universitaire congolais, avec la Faculté des Sciences de l'Université de Kinshasa comme cadre pilote.

Résultats obtenus

Le premier résultat majeur de ce travail est la conception et l'implémentation effective d'une plateforme web opérationnelle, intégrant un pipeline complet de traitement IA : extraction de texte (avec OCR pour les PDF scannés), prétraitement NLP, génération d'embeddings vectoriels par Sentence-BERT, recherche vectorielle via pgvector, calcul de similarité cosinus, classification des correspondances et génération automatique de rapports PDF. La plateforme est modularisée, conteneurisée avec Docker, et déployable sur une infrastructure réaliste dans le contexte congolais.

Le deuxième résultat majeur est la validation expérimentale de l'hypothèse formulée en introduction. L'évaluation quantitative conduite sur un corpus pilote de 150 travaux académiques démontre que l'approche sémantique fondée sur les embeddings Sentence-BERT surpasse significativement l'approche lexicale classique, avec un F1-score de 0,84 contre 0,66, soit un gain relatif de 27 %. Ce gain est particulièrement marqué sur la détection des paraphrases (rappel de 0,79 contre 0,28), des paraphrases produites par LLM (0,71 contre 0,12) et des traductions non sourcées (0,62 contre 0,05). L'approche hybride combinant méthodes lexicales et sémantiques atteint un F1-score de 0,87 et une AUC de 0,92.

Le troisième résultat est la démonstration de la pertinence d'une solution locale pour le contexte africain. La plateforme proposée répond aux limites des solutions commerciales existantes (Turnitin, Compilatio) : hébergement local des données, maîtrise du coût, adaptation au contexte linguistique francophone, prise en charge des spécificités du corpus académique local. L'architecture modulaire retenue ouvre la voie à une extension progressive à d'autres facultés et universités, sans refonte majeure.

Le quatrième résultat, plus qualitatif, est la contribution à la formation d'une nouvelle génération d'informaticiens congolais capables de concevoir, réaliser et maintenir des systèmes d'information complexes intégrant des technologies d'IA modernes. Le projet a mobilisé un large éventail de compétences : conception logicielle, programmation Python et TypeScript, administration de bases de données, apprentissage automatique, déploiement d'applications web, évaluation expérimentale.

Limites du travail

Plusieurs limites doivent être reconnues pour situer correctement la portée des résultats. La première concerne la taille du corpus pilote : 150 travaux académiques constituent un échantillon suffisant pour valider la preuve de concept et démontrer la supériorité de l'approche sémantique, mais restent limités pour généraliser les résultats à grande échelle. Une expérimentation sur plusieurs milliers de travaux, sur plusieurs années académiques et plusieurs facultés, sera nécessaire pour confirmer les performances en conditions réelles et pour affiner les seuils de décision.

La deuxième limite porte sur la détection du plagiat par IA générative. Avec un rappel de seulement 18 % sur les textes générés par ChatGPT, la plateforme ne répond que partiellement à ce défi émergent. Cette limite n'est pas spécifique à la solution proposée : aucun outil existant à ce jour n'offre une détection fiable des textes générés par les LLM modernes. La rapidité d'évolution de ces modèles (GPT-4, Claude 3, Gemini 1.5, etc.) rend toute approche de détection rapidement obsolète, et impose une veille permanente.

La troisième limite concerne les biais linguistiques des embeddings multilingues. Le modèle Sentence-BERT utilisé est entraîné sur un corpus majoritairement anglophone, et les similarités entre textes en français peuvent être affectées par des biais de distribution. Un fine-tuning sur un corpus académique francophone africain améliorerait la qualité des embeddings pour le contexte cible, mais nécessite des ressources de calcul non négligeables.

La quatrième limite est d'ordre infrastructurel : la plateforme a été testée sur un serveur de laboratoire, mais son déploiement à l'échelle d'une université nationale suppose une infrastructure stable (connectivité Internet, serveurs, stockage, sauvegardes) qui n'est pas toujours garantie dans le contexte congolais. Des solutions de déploiement dégradé (mode hors ligne, synchronisation différée) pourraient être étudiées.

La cinquième limite porte sur l'éthique et la confidentialité. Le traitement de travaux étudiants par une plateforme automatisée soulève des questions de protection des données,

de consentement, de durée de conservation, de droit à l'effacement. Ces questions, abordées dans la conception, nécessiteront un cadre juridique et organisationnel plus complet lors du déploiement opérationnel.

Perspectives

Les perspectives ouvertes par ce travail s'articulent autour de quatre axes complémentaires.

Extension à l'échelle nationale

La première perspective est l'extension progressive de la plateforme à l'ensemble des facultés de l'Université de Kinshasa, puis à d'autres universités congolaises. L'architecture modulaire retenue facilite cette extension : chaque faculté ou université peut être ajoutée comme un module indépendant, avec sa propre base de travaux, tout en bénéficiant du cœur technique commun. La constitution d'un référentiel académique national, regroupant les travaux de plusieurs établissements, ouvrirait la voie à une détection interuniversitaire du plagiat, particulièrement utile pour identifier les étudiants qui soumettent le même travail dans plusieurs établissements.

Amélioration de la détection

La deuxième perspective concerne l'amélioration des performances de détection. Plusieurs pistes peuvent être explorées : fine-tuning du modèle d'embeddings sur un corpus académique francophone, intégration d'approches spécialisées pour la détection du plagiat par IA générative (classificateurs supervisés, métriques de perplexité, analyse stylo-métrique), combinaison avec des approches citation analysis pour les travaux scientifiques, intégration de la détection multilingue au-delà du français et de l'anglais. L'évolution rapide du domaine imposera une veille technologique continue et des mises à jour régulières du pipeline IA.

Enrichissement fonctionnel

La troisième perspective porte sur l'enrichissement fonctionnel de la plateforme. Plusieurs évolutions sont envisageables : module de formation des étudiants à l'intégrité académique, avec auto-évaluation de leurs travaux avant dépôt ; intégration avec les systèmes de gestion scolaire existants (PGI, ENT) pour éviter la double saisie des

métadonnées ; API ouverte aux établissements partenaires pour permettre l'interopérabilité ; module de tableau de bord avancé pour le pilotage institutionnel de l'intégrité académique ; application mobile pour faciliter le dépôt et la consultation par les étudiants.

Recherche et publication

La quatrième perspective est la poursuite de la recherche sur les thèmes abordés. Le présent mémoire constitue une première contribution, qui appelle des approfondissements : publication des résultats d'évaluation dans des revues scientifiques du domaine, participation aux campagnes d'évaluation internationales (PAN, SemEval), collaboration avec d'autres équipes de recherche africaines travaillant sur des problématiques similaires, encadrement de mémoires complémentaires sur des aspects spécifiques (détection du plagiat par IA générative, fine-tuning multilingue, etc.).

Au-delà de ces axes, le présent travail s'inscrit dans une dynamique plus large de renforcement des capacités numériques locales et de contribution de la recherche africaine aux grands défis technologiques contemporains. La détection du plagiat académique n'est qu'un exemple des nombreux domaines où l'application des techniques modernes d'Intelligence Artificielle peut apporter des solutions pertinentes aux défis spécifiques du contexte congolais et africain. Le développement de solutions locales, adaptées, maîtrisées et évolutives, constitue une alternative nécessaire à la dépendance technologique extérieure, et participe à la construction d'une souveraineté numérique nationale.

BIBLIOGRAPHIE

Ouvrages et manuels

Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2018). BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. arXiv preprint arXiv:1810.04805.

Goodfellow, I., Bengio, Y., & Courville, A. (2016). Deep Learning. MIT Press.

Jurafsky, D., & Martin, J. H. (2024). Speech and Language Processing (3rd ed., draft). <https://web.stanford.edu/~jurafsky/slp3/>

Mikolov, T., Sutskever, I., Chen, K., Corrado, G., & Dean, J. (2013). Distributed Representations of Words and Phrases and their Compositionality. Advances in Neural Information Processing Systems, 26.

Pennington, J., Socher, R., & Manning, C. D. (2014). GloVe: Global Vectors for Word Representation. Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP), 1532–1543.

Reimers, N., & Gurevych, I. (2019). Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP), 3982–3992.

Sutton, R. S., & Barto, A. G. (2018). Reinforcement Learning: An Introduction (2nd ed.). MIT Press.

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., & Polosukhin, I. (2017). Attention Is All You Need. Advances in Neural Information Processing Systems, 30.

Articles et publications scientifiques

Agarwal, B., et al. (2023). Detecting AI-Generated Text: Factors Influencing Human Accuracy. arXiv preprint arXiv:2306.03229.

Bensalem, I., et al. (2019). Plagiarism Detection in Academic Writings: A Systematic Review. Computing Reviews.

Gipp, B., & Beel, J. (2010). Citation Pattern Matching Algorithms for Citation-based Plagiarism Detection. Proceedings of the 5th International Plagiarism Conference.

Potthast, M., et al. (2010). Overview of the 1st International Competition on Plagiarism Detection. SEPLN.

Stein, B., & zu Eissen, S. M. (2006). Near Similarity Search and Plagiarism Analysis. Studies in Classification, Data Analysis, and Knowledge Organization.

Weber-Wulff, D. (2014). False Positives in Plagiarism Detection. Plagiarism Across Europe and Beyond Conference Proceedings.

Documentation technique

FastAPI Documentation. <https://fastapi.tiangolo.com/>

pgvector: Open-source vector similarity search for PostgreSQL.
<https://github.com/pgvector/pgvector>

PostgreSQL Documentation (version 16). <https://www.postgresql.org/docs/16/>

React Documentation. <https://react.dev/>

Sentence-Transformers Documentation. <https://www.sbert.net/>

Tesseract OCR Documentation. <https://tesseract-ocr.github.io/>

spaCy Industrial-Strength Natural Language Processing. <https://spacy.io/>

Docker Documentation. <https://docs.docker.com/>

Sources académiques et institutionnelles

Université de Kinshasa. (2023). Règlement intérieur de la Faculté des Sciences.

UNESCO. (2023). L'éthique de l'intelligence artificielle. Recommandations.

Ministère de l'Enseignement Supérieur et Universitaire de la RDC. (2022). Cadre normatif des études supérieures.

Ressources en ligne

Copy, Shake, and Paste Blog. <https://copy-shake-paste.blogspot.com/> (consulté en 2024).

Hugging Face Model Hub. <https://huggingface.co/models> (consulté en 2024).

PAN: Plagiarism, Authorship, and Social Software Misuse. <https://pan.webis.de/> (consulté en 2024).